

DEPARTMENT OF INFORMATION TECHNOLOGY

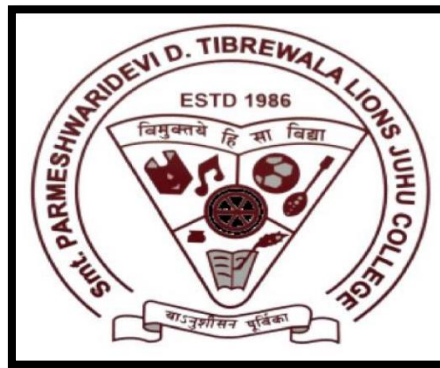
SMT. PARMESHWARIDEVI DURGADUTT TIBREWALA

LIONS JUHU COLLEGE

OF ARTS, COMMERE AND SCIENCE

Affiliated to University of Mumbai

J.B. NAGAR, ANDHERI (E), MUMBAI-400059



Academic Year 2022-2023

RESEARCH IN COMPUTING

For

Semester I

Submitted By:

Ms. KIRTI YADAV

Msc.IT (Sem I)

SMT. PARMESHWARIDEVI DURGADUTT TIBREWALA

LIONS JUHU COLLEGE

OF ARTS, COMMERE AND SCIENCE

Affiliated to University of Mumbai

J.B. NAGAR, ANDHERI (E), MUMBAI-400059

DEPARTMENT OF INFORMATION TECHNOLOGY



Certificate of Approval

This is to certify that practical entitled **“RESEARCH IN COMPUTING “**, Undertaken at **SMT.PARMESHWARIDEVI DURGADUTT TIBREWALA LIONS JUHU COLLEGE OF ARTS, COMMERECE & SCIENCE**. By **Ms. YADAV KIRTI RAMVILASH** Seat No. _____ in partial fulfilment of **M.Sc. (IT) master degree (Semester I)** Examination had not been submitted for any other examination and does not form of any other course undergone by the candidate. It is further certified that she has completed all required phases of the practical.

Internal Examiner

External Examiner

HOD / In-Charge / Coordinator

**Signature/
Principal/Stamp**

INDEX

Sr No.	Name of the Practical	Page No.	Date	Teacher's Signature
1A	Write a program for obtaining descriptive statistics of data.	01	04-10-2020	
1B	Import data from different data sources (from Excel, CSV, MySQL, SQL Server, Oracle to R/Python/Excel).	03	06-10-2022	
2A	Design a survey form for a given case study, collect the primary data and analyse it.	07	13-10-2022	
2B	Perform suitable analysis of given secondary data.	09	01-11-2022	
3A	Perform testing of hypothesis using one sample t-test.	12	03-11-2022	
3B	Perform testing of hypothesis using two sample t-test.	14	08-11-2022	
3C	Perform testing of hypothesis using paired t-test.	16	08-11-2022	
4A	Perform testing of hypothesis using chi-squared goodness-of-fit test.	19	15-11-2022	
4B	Perform testing of hypothesis using chi-squared Test of Independence.	21	15-11-2022	
5	Perform testing of hypothesis using Z-test.	23	22-11-2022	
6A	Perform testing of hypothesis using one-way ANOVA.	26	01-12-2022	
6B	Perform testing of hypothesis using two-way ANOVA.	30	01-12-2022	
6C	Perform testing of hypothesis using multivariate ANOVA (MANOVA).	34	07-12-2022	
7A	Perform the Random sampling for the given data and analyse it.	36	07-12-2022	
7B	Perform the Stratified sampling for the given data and analyse it.	38	10-12-2022	

8	Compute different types of correlation.	42	10-12-2022	
9A	Perform linear regression for prediction.	46	13-12-2022	
9B	Perform polynomial regression for prediction.	49	13-12-2022	
10A	Perform multiple linear regression.	52	17-12-2022	
10B	Perform Logistic regression.	56	17-12-2022	

PRACTICAL 1**A. Aim:- Write a program for obtaining descriptive statistics of data.****Code**

```
import pandas as pd
#Create a Dictionary of series
d = {'Age':pd.Series([25,26,25,23,30,29,23,34,40,30,51,46]),
'Rating':pd.Series([4.23,3.24,3.98,2.56,3.20,4.6,3.8,3.78,2.98,4.80,4.10,3.65])}
#Create a DataFrame
df = pd.DataFrame(d)
print(df)
print('##### Sum ##### ')
print (df.sum())
print('##### Mean ##### ')
print (df.mean())
print('##### Standard Deviation ##### ')
print (df.std())
print('##### Descriptive Statistics ##### ')
print (df.describe())
```

Output

```

IDLE Shell 3.10.0
File Edit Shell Debug Options Window Help
Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information
>>>
= RESTART: C:\College\KIRTI\Msc IT Part 1 Sem 1\Practicals\RESEARCH IN
\Python File\PRACTICAL 1A.py
    Age    Rating
0      25      4.23
1      26      3.24
2      25      3.98
3      23      2.56
4      30      3.20
5      29      4.60
6      23      3.80
7      34      3.78
8      40      2.98
9      30      4.80
10     51      4.10
11     46      3.65
##### Sum #####
Age      382.00
Rating   44.92
dtype: float64
##### Mean #####
Age      31.833333
Rating   3.743333
dtype: float64
##### Standard Deviation #####
Age      9.232682
Rating   0.661628
dtype: float64
##### Descriptive Statistics #####
      Age    Rating
count  12.000000  12.000000
mean   31.833333   3.743333
std     9.232682   0.661628
min    23.000000   2.560000
25%    25.000000   3.230000
50%    29.500000   3.790000
75%    35.500000   4.132500
  
```

B. Aim:- Import data from different data sources (from excel, csv, mysql, sql server, oracle to r/python/excel)**SQLite**

```
import sqlite3 as sq
import pandas as pd

sDatabaseName='C:/Users/Ravi yadav/Downloads/chinook (1).db'

conn = sq.connect(sDatabaseName)

#####

sFileName='C:/VKHCG/01-Vermeulen/01-Retrieve/01-EDS/02-
Python/Retrieve_IP_DATA.csv'

print('Loading :',sFileName)

IP_DATA_ALL_FIX=pd.read_csv(sFileName,header=0,low_memory=False)

IP_DATA_ALL_FIX.index.names = ['RowIDCSV']

sTable='IP_DATA_ALL'

print('Storing :',sDatabaseName,' Table:',sTable)

IP_DATA_ALL_FIX.to_sql(sTable, conn, if_exists="replace")

print('Loading :',sDatabaseName,' Table:',sTable)

TestData=pd.read_sql_query("select * from IP_DATA_ALL;", conn)

print('#####')

print('## Data Values')

print('#####')

print(TestData)

print('#####')

print('## Data Profile')

print('#####')

print('Rows :',TestData.shape[0])

print('Columns :',TestData.shape[1])

print('#####')

print('### Done!! #####')
```

Output

```
= RESTART: C:\College\KIRTI\Msc IT Part 1 Sem 1\Practicals\RESEARCH IN COMPUTING\Python File\lb.py
Loading : C:\VKHCG\01-Vermeulen\01-Retrieve\01-EDS\02-Python\Retrieve_IP_DATA.csv
Storing : C:/Users/Ravi yadav/Downloads/chinook (1).db Table: IP_DATA_ALL
Loading : C:/Users/Ravi yadav/Downloads/chinook (1).db Table: IP_DATA_ALL
#####
## Data Values
#####
      RowIDCSV  RowID  ID  ... Longitude First.IP.Number Last.IP.Number
0           0      0   1  ...   25.9119      692781056      692781567
1           1      1   2  ...   25.9119      692781824      692783103
2           2      2   3  ...   25.9119      692909056      692909311
3           3      3   4  ...   25.9119      692909568      692910079
4           4      4   5  ...   25.9119      693051392      693052415
..         ...    ...  ..  ...         ...         ...
66          66     66  67  ...   -0.2167      692253824      692253951
67          67     67  68  ...   -0.2167      692254208      692254719
68          68     68  69  ...   -0.2167      692256000      692256255
69          69     69  70  ...   -0.2167      692738048      692740095
70          70     70  71  ...   -0.2167      692852736      692852991

[71 rows x 10 columns]
#####
## Data Profile
#####
Rows : 71
Columns : 10
#####
### Done!! #####
|
```


Microsoft Excel

```
#####

import os

import pandas as pd

#####

Base='C:/VKHCG'

#####

sFileDir=Base + '/01-Vermeulen/01-Retrieve/01-EDS/02-Python'

#if not os.path.exists(sFileDir):

#os.makedirs(sFileDir)

#####

CurrencyRawData = pd.read_excel('C:/VKHCG/01-Vermeulen/00-
RawData/Country_Currency.xlsx')

sColumns = ['Country or territory', 'Currency', 'ISO-4217']

CurrencyData = CurrencyRawData[sColumns]

CurrencyData.rename(columns={'Country or territory': 'Country', 'ISO-4217':
'CurrencyCode'}, inplace=True)

CurrencyData.dropna(subset=['Currency'],inplace=True)

CurrencyData['Country'] = CurrencyData['Country'].map(lambda x: x.strip())

CurrencyData['Currency'] = CurrencyData['Currency'].map(lambda x:
x.strip())

CurrencyData['CurrencyCode'] = CurrencyData['CurrencyCode'].map(lambda x:
x.strip())

print(CurrencyData)

print('~~~~~ Data from Excel Sheet Retrived Successfully ~~~~~ ')

#####

sFileName=sFileDir + '/Retrieve-Country-Currency.csv'

CurrencyData.to_csv(sFileName, index = False)

#####
```

Output

```
= RESTART: C:\College\KIRTI\Msc IT Part 1 Sem 1\Practicals\RESEARCH IN COMPUTING
\Python File\prac 1b 3rd.py
      Country      Currency CurrencyCode
1      Afghanistan  Afghan afghani      AFN
2  Akrotiri and Dhekelia (UK)  European euro      EUR
3      Aland Islands (Finland)  European euro      EUR
4      Albania      Albanian lek      ALL
5      Algeria      Algerian dinar      DZD
..      ...      ...      ...
271      Wake Island (USA)  United States dollar      USD
272  Wallis and Futuna (France)  CFP franc      XPF
274      Yemen      Yemeni rial      YER
276      Zambia      Zambian kwacha      ZMW
277      Zimbabwe  United States dollar      USD

[253 rows x 3 columns]
~~~~~ Data from Excel Sheet Retrived Successfully ~~~~~
```

PRACTICAL 2

A. Aim:- Design a survey form for a given case study, collect the primary data and analyse it.

Case 1:

A researcher wants to conduct a Survey in colleges on Use of ICT in higher education from Mumbai, Thane and Navi Mumbai. The survey focuses on access to and use of ICT in teaching and learning, as well as on attitudes towards the use of ICT in teaching and learning.

Design questionnaire addressed to teachers seeks information about the target class, his experience using ICT for teaching, access to ICT infrastructure, support available, ICT based activities and material used, obstacles to the use of ICT in teaching, learning activities with the target class, your skills and attitudes to ICT, and some personal background information.

Arrange question in following groups:

1. Information about the target class you teach
2. Experience with ICT for teaching
3. ICT access for teaching
4. Support to teachers for ICT use
5. ICT based activities and material used for teaching
6. Obstacles to using ICT in teaching and learning
7. Learning activities with the target class
8. Teacher skills
9. Teacher opinions and attitudes
10. Personal background information

Case 2:

A research agency wants to study the perception about App based taxi service in Mumbai, Thane and Navi Mumbai. The survey focuses on customers attitude towards app base taxi service as well as on attitudes towards regular taxi cab. Design questionnaire seeks information about the target taxi service, his experience using taxi services, access, support available, obstacles and some personal background information, with the following objectives:

1. To find out the customer satisfaction towards the App based-taxi services.
2. To find the level of convenience and comfort with App based -taxi services.
3. To know their opinion about the tariff system and promptness of service.
4. To ascertain the customer view towards the driver behaviour and courtesy.

5. To provide inputs to enhance the services to delight the customers.
6. To examine relationship between service quality factors and taxi passenger satisfaction.
7. To suggest better regulations for transportation authorities regarding customer protection and effective monitoring of taxi services.

Case 3:

A popular electronic store wants to conduct a survey to develop awareness of branded laptop baseline estimates and determine popularity of different company's laptop. It suggests steps to be initiated or strengthened in the field of demand in a region. The key indicators are among the general population, demand branded laptop and the problem users.

The objectives of this particular study are:-

1. To know the preferences of different types of branded laptops by students and professionals.
2. To study which factor influence for choosing different types of branded laptops.
3. To know about the level of satisfaction towards different types of branded laptops.
4. To identify the perception of consumers towards the laptop positioning strategy.
5. To know the consumer preference towards laptop in the present era.

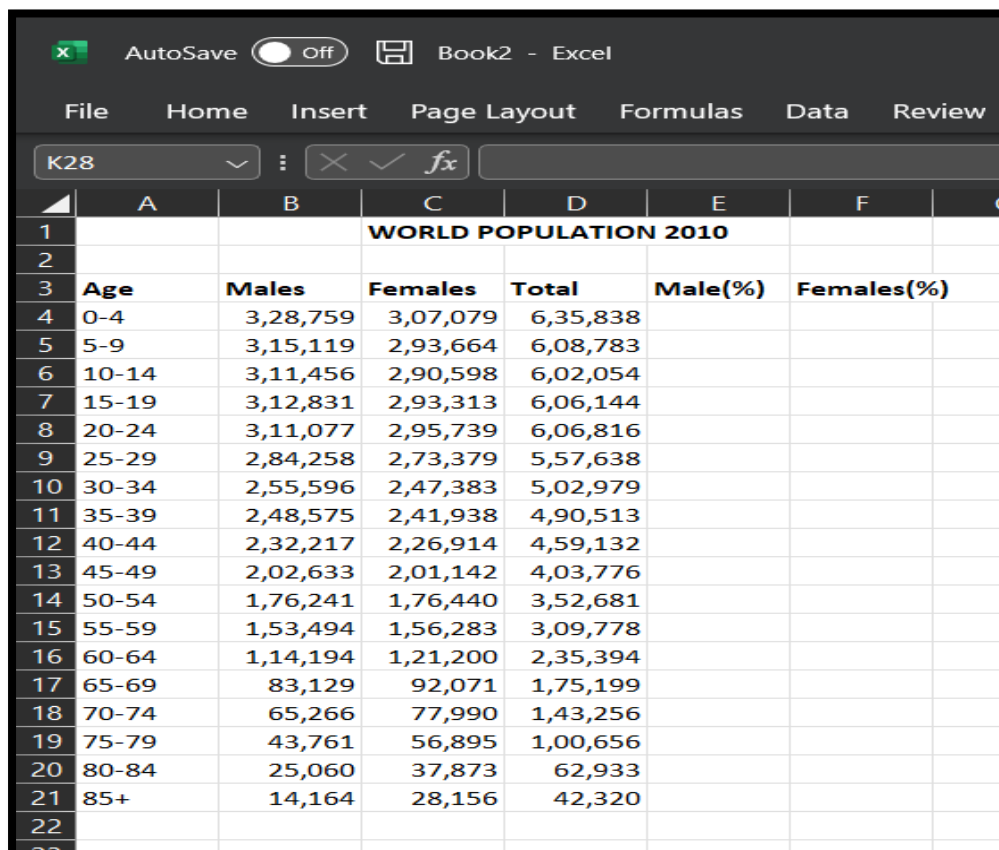
Use the collected data for analysis.

B. Aim:- Perform analysis of given secondary data.

Steps in Secondary Data Analysis:

- 1. Determine your research question** – Knowing exactly what you are looking for.
- 2. Locating data**– Knowing what is out there and whether you can gain access to it. A quick Internet search, possibly with the help of a librarian, will reveal a wealth of options.
- 3. Evaluating relevance of the data** – Considering things like the data's original purpose, when it was collected, population, sampling strategy/sample, data collection protocols, operationalization of concepts, questions asked, and form/shape of the data.
- 4. Assessing credibility of the data** – Establishing the credentials of the original researchers, searching for full explication of methods including any problems encountered, determining how consistent the data is with data from other sources, and discovering whether the data has been used in any credible published research.
- 5. Analysis** – This will generally involve a range of statistical processes.

Example: Analyze the given Population Census Data for Planning and Decision Making by using the size and composition of populations.



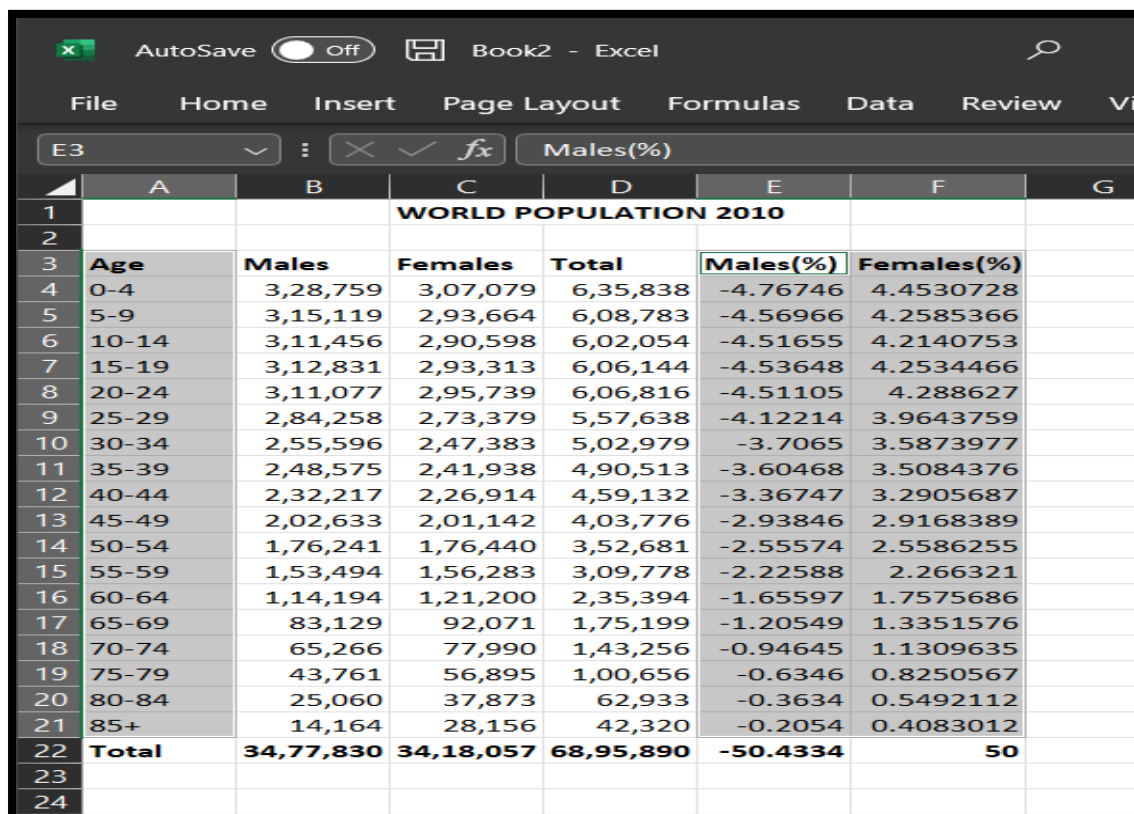
The screenshot shows an Excel spreadsheet titled 'Book2 - Excel'. The active sheet is 'K28'. The data is organized in a table with columns for Age, Males, Females, Total, Male(%), and Females(%). The rows represent age groups from 0-4 to 85+.

	A	B	C	D	E	F
1		WORLD POPULATION 2010				
2						
3	Age	Males	Females	Total	Male(%)	Females(%)
4	0-4	3,28,759	3,07,079	6,35,838		
5	5-9	3,15,119	2,93,664	6,08,783		
6	10-14	3,11,456	2,90,598	6,02,054		
7	15-19	3,12,831	2,93,313	6,06,144		
8	20-24	3,11,077	2,95,739	6,06,816		
9	25-29	2,84,258	2,73,379	5,57,638		
10	30-34	2,55,596	2,47,383	5,02,979		
11	35-39	2,48,575	2,41,938	4,90,513		
12	40-44	2,32,217	2,26,914	4,59,132		
13	45-49	2,02,633	2,01,142	4,03,776		
14	50-54	1,76,241	1,76,440	3,52,681		
15	55-59	1,53,494	1,56,283	3,09,778		
16	60-64	1,14,194	1,21,200	2,35,394		
17	65-69	83,129	92,071	1,75,199		
18	70-74	65,266	77,990	1,43,256		
19	75-79	43,761	56,895	1,00,656		
20	80-84	25,060	37,873	62,933		
21	85+	14,164	28,156	42,320		
22						

Put the cursor in **cell B22** and click on the **AutoSum** and then click **Enter**. This will calculate the total population. Then copy the formula in **cell D22** across the **row 22**.

To calculate the percent of males in **cell E4**, enter the formula $=-1*100*B4/D$22$. And copy the formula in **cell E4** down to **cell E21**.

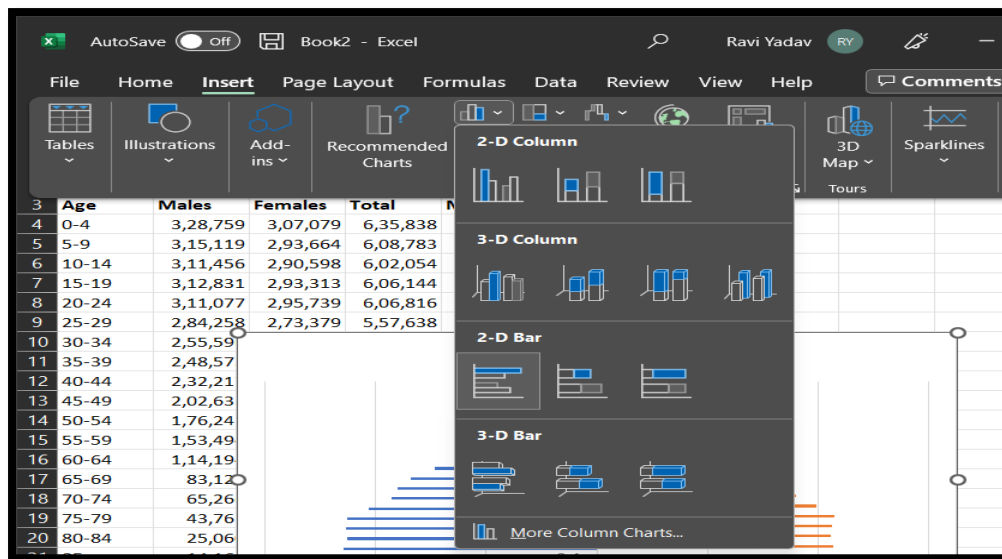
To calculate the percent of females in **cell F4**, enter the formula $=100*C4/D$22$. Copy the formula in **cell F4** down to **cell F21**.



	A	B	C	D	E	F	G
1			WORLD POPULATION 2010				
2							
3	Age	Males	Females	Total	Males(%)	Females(%)	
4	0-4	3,28,759	3,07,079	6,35,838	-4.76746	4.4530728	
5	5-9	3,15,119	2,93,664	6,08,783	-4.56966	4.2585366	
6	10-14	3,11,456	2,90,598	6,02,054	-4.51655	4.2140753	
7	15-19	3,12,831	2,93,313	6,06,144	-4.53648	4.2534466	
8	20-24	3,11,077	2,95,739	6,06,816	-4.51105	4.288627	
9	25-29	2,84,258	2,73,379	5,57,638	-4.12214	3.9643759	
10	30-34	2,55,596	2,47,383	5,02,979	-3.7065	3.5873977	
11	35-39	2,48,575	2,41,938	4,90,513	-3.60468	3.5084376	
12	40-44	2,32,217	2,26,914	4,59,132	-3.36747	3.2905687	
13	45-49	2,02,633	2,01,142	4,03,776	-2.93846	2.9168389	
14	50-54	1,76,241	1,76,440	3,52,681	-2.55574	2.5586255	
15	55-59	1,53,494	1,56,283	3,09,778	-2.22588	2.266321	
16	60-64	1,14,194	1,21,200	2,35,394	-1.65597	1.7575686	
17	65-69	83,129	92,071	1,75,199	-1.20549	1.3351576	
18	70-74	65,266	77,990	1,43,256	-0.94645	1.1309635	
19	75-79	43,761	56,895	1,00,656	-0.6346	0.8250567	
20	80-84	25,060	37,873	62,933	-0.3634	0.5492112	
21	85+	14,164	28,156	42,320	-0.2054	0.4083012	
22	Total	34,77,830	34,18,057	68,95,890	-50.4334	50	
23							
24							

To build the population pyramid, we need to choose a horizontal bar chart with two series of data (% male and % female) and the age labels in column A as the **Category X-axis** labels. Highlight the range **A3:A21**, hold down the CTRL key and highlight the range **E3:F21**.

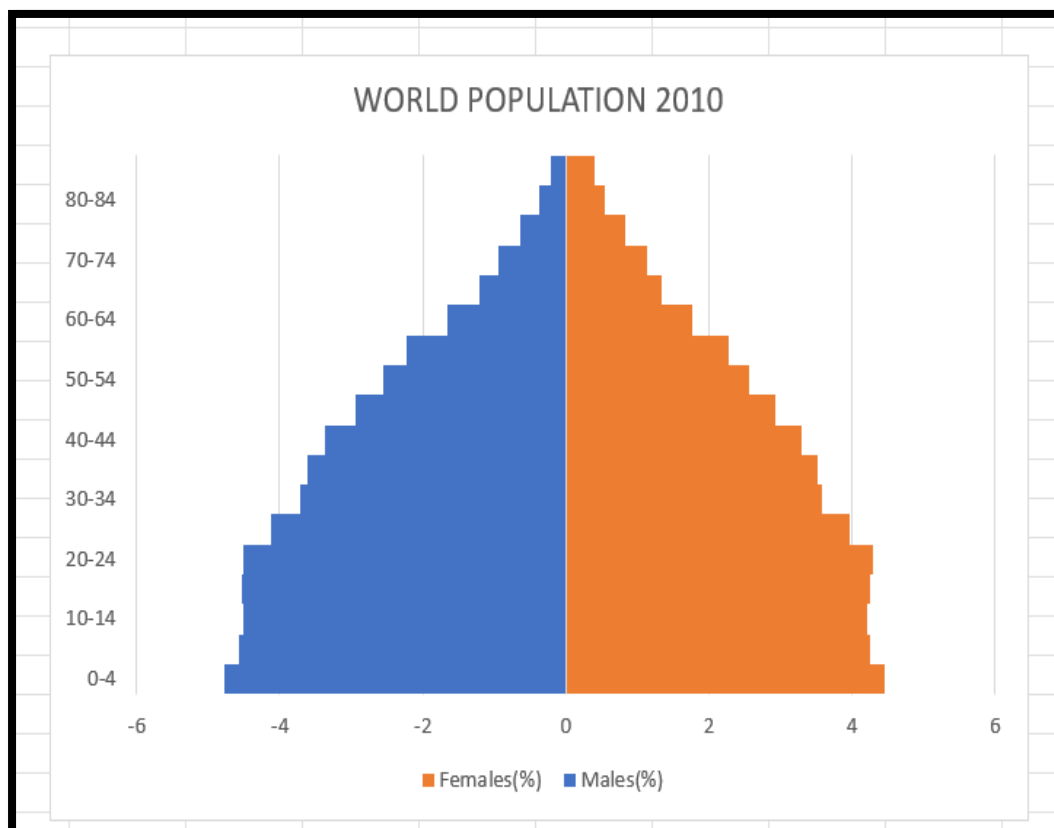
Under **inset** tab, under horizontal bar charts select **clustered bar chart**.



Put the tip of your mouse arrow on the **Y-axis** (vertical axis) so it says “Category Axis”, right click and chose **Format Axis**

Choose **Axis options** tab and set the major and minor tick mark type to **None**, Axis labels to **Low**, and click **OK**.

Click on any of the bars in your pyramid, click right and select “format data series”. Set the **Overlap** to **100** and **Gap Width** to **0**. Click **OK**.



PRACTICAL 3

A. Aim:- Perform testing of hypothesis using one sample t-test.

One sample t-test : The One Sample t Test determines whether the sample mean is statistically different from a known or hypothesised population mean. The One Sample t Test is a parametric test.

Code

```
from scipy.stats import ttest_1samp
import numpy as np
ages = np.genfromtxt('ages.csv')
print(ages)
ages_mean = np.mean(ages)
print(ages_mean)
tset, pval = ttest_1samp(ages, 30)
print('p-values - ',pval)
if pval< 0.05:
    # alpha value is 0.05
    print(" we are rejecting null hypothesis")
else:
    print("we are accepting null hypothesis")
```


Output

```
= RESTART: C:\College\KIRTI\Msc IT Part 1 Sem 1\Practicals\RESEARCH IN COMPUTING
\Python File\PRACTICAL 3A.py
[20. 30. 25. 13. 16. 17. 34. 35. 38. 42. 43. 45. 48. 49. 50. 51. 54. 55.
 56. 59. 61. 62. 18. 22. 29. 30. 31. 39. 52. 53. 67. 36. 47. 54. 40. 40.
 35. 22. 59. 58. 30. 43. 22. 45. 21. 59. 51. 47. 25. 58. 50. 23. 24. 45.
 37. 59. 28. 28. 48. 42. 54. 36. 36. 24. 26. 24. 50. 48. 34. 44. 56. 55.
 35. 33. 39. 53. 34. 28. 56. 24. 21. 29. 28. 58. 35. 57. 26. 25. 59. 56.
 22. 57. 48. 33. 23. 26. 57. 32. 53. 31. 35. 44. 54. 25. 31. 58. 26. 32.
 26. 50. 41. 49. 26. 33. 34. 24. 43. 42. 51. 36. 38. 38. 40. 38. 56. 39.
 23. 33. 53. 30. 38.]
39.47328244274809
p-values - 5.362905195437013e-14
we are rejecting null hypothesis
```

B. Aim:- Write a program for t-test comparing two means for independent samples.

The t distribution provides a good way to perform one sample tests on the mean when the population variance is not known provided the population is normal or the sample is sufficiently large so that the Central Limit Theorem applies.

Two Sample t-Test**Code**

```
import numpy as np
from scipy import stats
from numpy.random import randn
N = 20
#a = [35,40,12,15,21,14,46,10,28,48,16,30, 32,48,31,22,12,39,19,25]
#b = [2,27,31,38,1,19,1,34,3,1,2,1,3,1,2,1,3,29,37,2]
a = 5 * randn(100) + 50
b = 5 * randn(100) + 51
var_a = a.var(ddof=1)
var_b = b.var(ddof=1)
s = np.sqrt((var_a + var_b)/2)
t = (a.mean() - b.mean())/(s*np.sqrt(2/N))
df = 2*N - 2
#p-value after comparison with the t
p = 1 - stats.t.cdf(t,df=df)
print("t = " + str(t))
print("p = " + str(2*p))
if t > p :
    print('Mean of two distribution are differnt and significant')
else:
    print('Mean of two distribution are same and not significant')
```

Output

```
= RESTART: C:\College\KIRTI\Msc IT Part 1 Sem 1\Practicals\RESEARCH IN COMPUTING  
\Python File\PRACTICAL 3B.py  
t = -0.947144521675142  
p = 1.6504503777005022  
Mean of two distribution are same and not significant
```

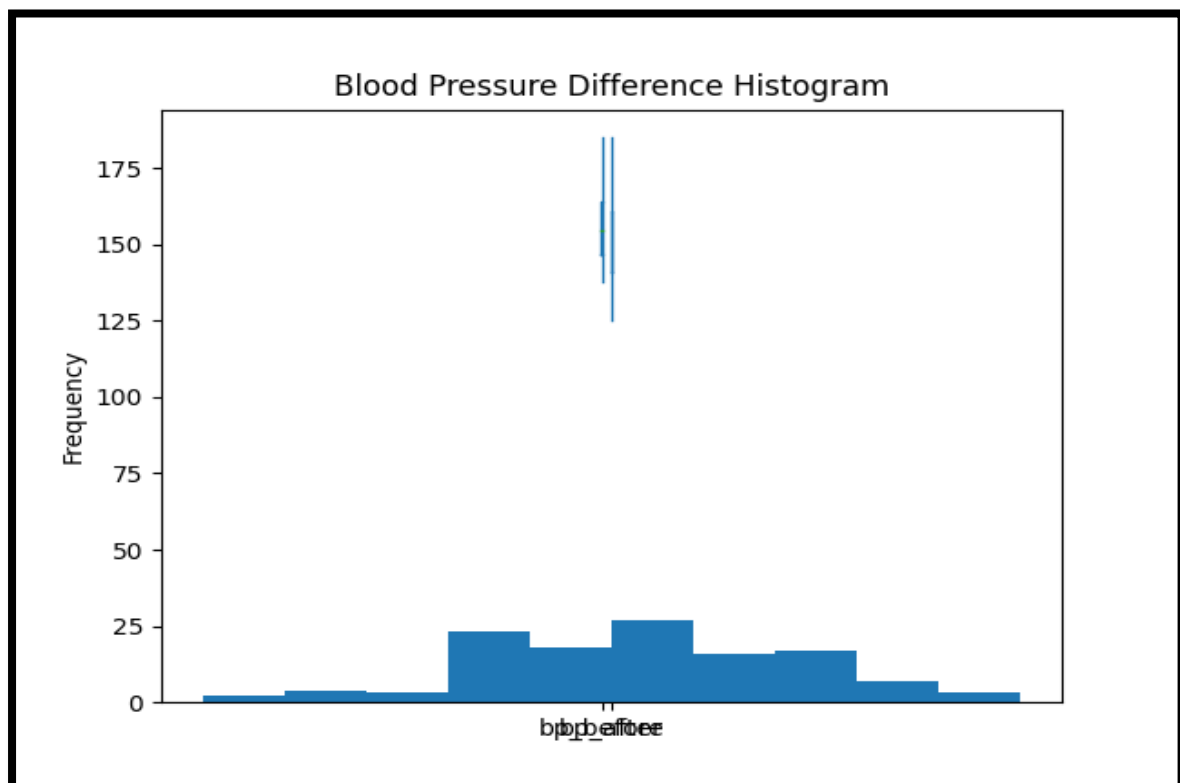
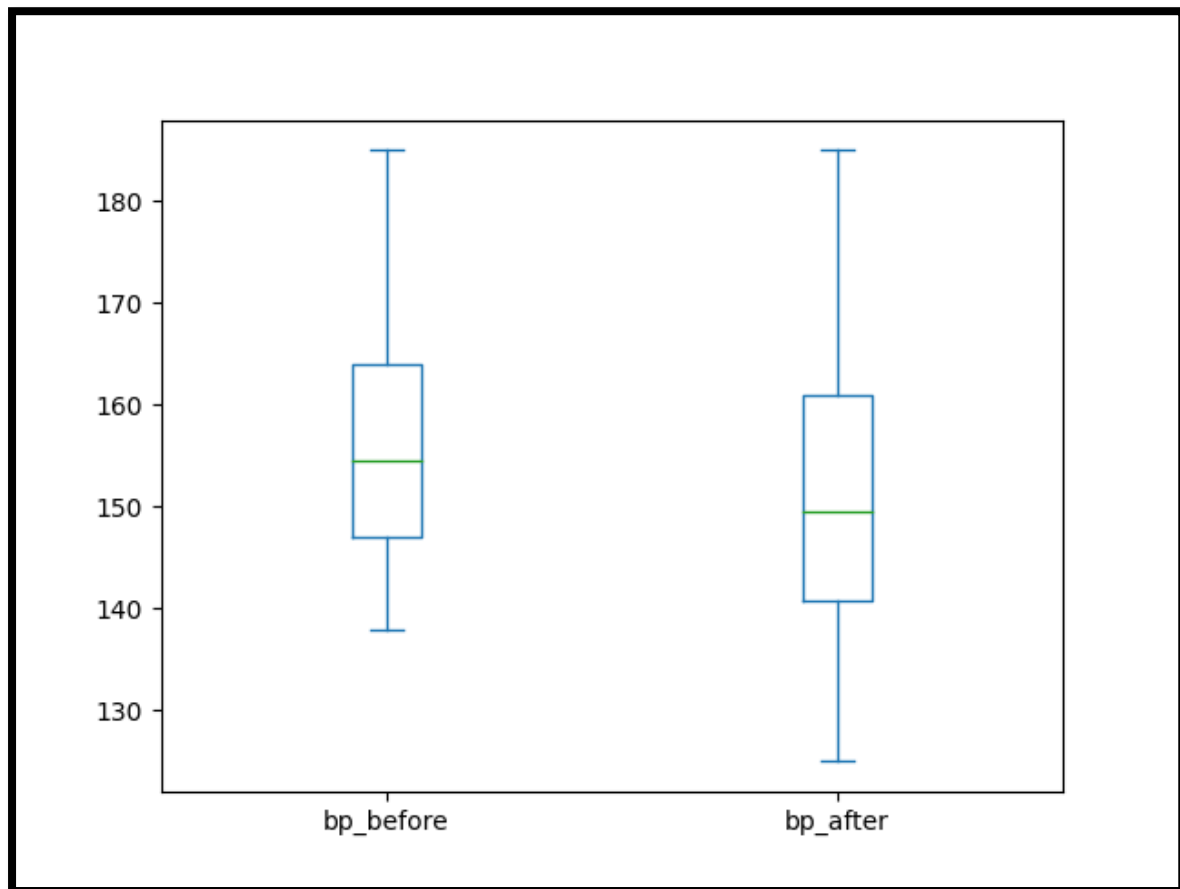
C. Aim:- Perform testing of hypothesis using paired t-test.

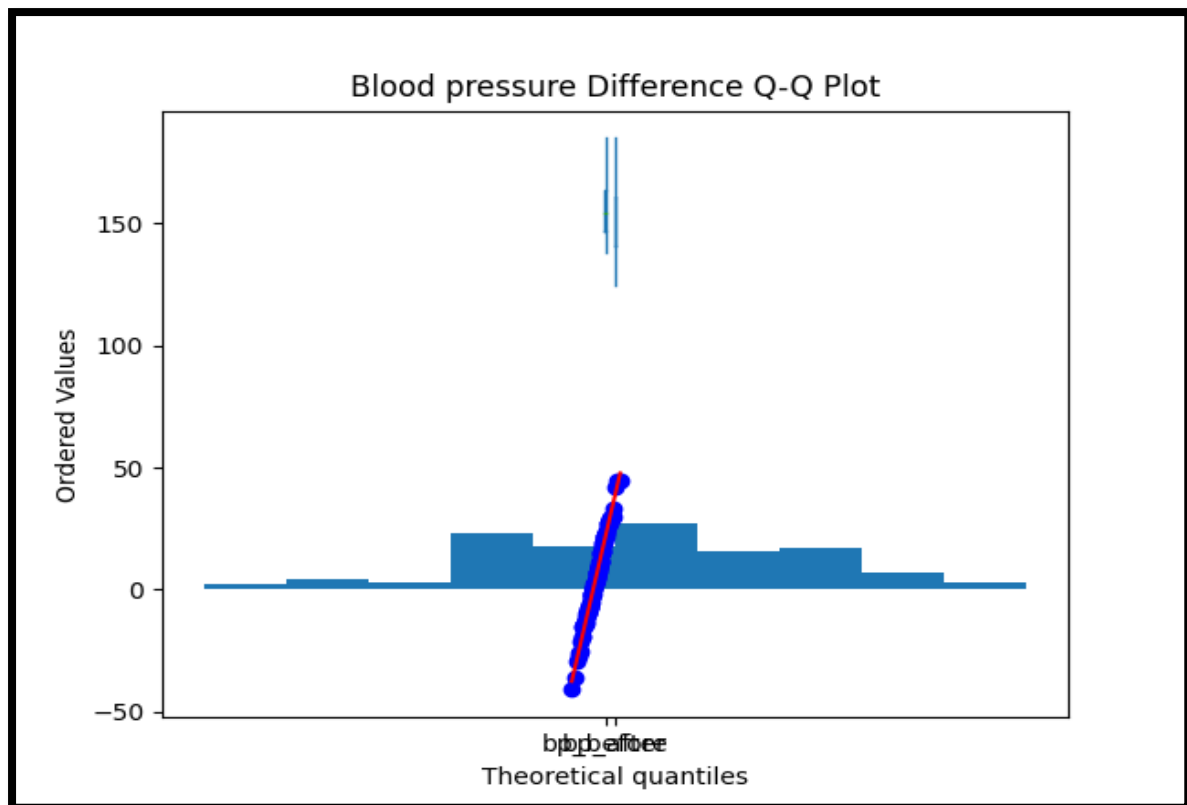
The paired sample t-test is also called dependent sample t-test. It's an univariate test that tests for a significant difference between 2 related variables. An example of this is if you were to collect the blood pressure for an individual before and after some treatment, condition, or time point. The data set contains blood pressure readings before and after an intervention. These are variables "bp_before" and "bp_after".

The hypothesis being test is:

- **H₀** - The mean difference between sample 1 and sample 2 is equal to 0.
- **H₁** - The mean difference between sample 1 and sample 2 is not equal to 0

```
from scipy import stats
import matplotlib.pyplot as plt
import pandas as pd
df = pd.read_csv("blood_pressure.csv")
print(df[['bp_before', 'bp_after']].describe())
#First let's check for any significant outliers in
#each of the variables.
df[['bp_before', 'bp_after']].plot(kind='box')
# This saves the plot as a png file
plt.savefig('boxplot_outliers.png')
# make a histogram to differences between the two scores.
df['bp_difference'] = df['bp_before'] - df['bp_after']
df['bp_difference'].plot(kind='hist', title= 'Blood Pressure Difference Histogram')
#Again, this saves the plot as a png file
plt.savefig('blood pressure difference histogram.png')
stats.probplot(df['bp_difference'], plot= plt)
plt.title('Blood pressure Difference Q-Q Plot')
plt.savefig('blood pressure difference qq plot.png')
stats.shapiro(df['bp_difference'])
stats.ttest_rel(df['bp_before'], df['bp_after'])
```

Output



```

IDLE Shell 3.10.0
File Edit Shell Debug Options Window Help
Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\College\KIRTI\Msc IT Part 1 Sem 1\Practicals\RESEARCH IN COMPUTING
\Python File\PRACTICAL 3C.py
      bp_before  bp_after
count  120.000000  120.000000
mean   156.450000  151.358333
std     11.389845   14.177622
min     138.000000  125.000000
25%     147.000000  140.750000
50%     154.500000  149.500000
75%     164.000000  161.000000
max     185.000000  185.000000
>>>

```

A paired sample t-test was used to analyze the blood pressure before and after the intervention to test if the intervention had a significant affect on the blood pressure. The blood pressure before the intervention was higher (156.45 ± 11.39 units) compared to the blood pressure post intervention (151.36 ± 14.18 units); there was a statistically significant decrease in blood pressure ($t(119)=3.34$, $p=0.0011$) of 5.09 units.

PRACTICAL 4

A. Aim:- Perform testing of hypothesis using chi-squared goodness-of-fit test.

Problem :

An system administrator needs to upgrade the computers for his division. He wants to know what sort of computer system his workers prefer. He gives three choices: Windows, Mac, or Linux. Test the hypothesis or theory that an equal percentage of the population prefers each type of computer system .

	A	B	C	D	E
1	System	O	Ei	$\Sigma(O_i - E_i)^2 / E_i$	
2	Windows	20	33.33		
3	Mac	60	33.33		
4	Linux	20	33.33		
5	Total	100	99.99		
6					

H₀ : The population distribution of the variable is the same as the proposed distribution

H₁ : The distributions are different

To calculate the Chi –Squared value for Windows go to **cell D2** and type

$$=((B2- C2)*(B2-C2))/C2$$

To calculate the Chi –Squared value for Mac go to **cell D3** and type

$$=((B3-C3)*(B3- C3))/C3$$

To calculate the Chi –Squared value for Mac go to **cell D3** and type

$$=((B4-C4)*(B4- C4))/C4$$

Go to **Cell D5** for $\frac{\Sigma(O_i - E_i)^2}{E_i}$ and type =SUM(D2:D4)

To get the table value for Chi-Square for $\alpha = 0.05$ and dof = 2, go to **cell D7** and type

$$=CHIINV(0.05,2)$$

At **cell D9** type =IF(D5>D7, "H₀ Accepted","H₀ Rejected")

Output

AutoSave Off PRACTICAL 4A Search (Alt+Q) FileHomeInsertPage LayoutFormulasDataReviewViewHelp D10 ✕ ✓ f_x													
	A	B	C	D	E	F	G	H	I	J	K	L	M
1	System	O	Ei	$\Sigma(O_i - E_i)^2 / E_i$									
2	Windows	20	33.33	5.33120012		H0 : The population distribution of the variable is the same as the proposed distribution							
3	Mac	60	33.33	21.3408011		H1 : The distributions are different							
4	Linux	20	33.33	5.33120012									
5	Total	100	99.99	32.0032013									
6													
7			Table Value	5.99146455									
8													
9				H0 Accepted									
10													

B. Aim:- Perform testing of hypothesis using chi-squared test of independence.**Code**

```
import numpy as np
import pandas as pd
import scipy.stats as stats
np.random.seed(10)
stud_grade = np.random.choice(a=["O","A","B","C","D"],
p=[0.20, 0.20 ,0.20, 0.20, 0.20], size=100)
stud_gen = np.random.choice(a=["Male","Female"], p=[0.5, 0.5], size=100)
mscpart1 = pd.DataFrame({"Grades":stud_grade, "Gender":stud_gen})
print(mscpart1)
stud_tab = pd.crosstab(mscpart1.Grades, mscpart1.Gender, margins=True)
stud_tab.columns = ["Male", "Female", "row_totals"]
stud_tab.index = ["O", "A", "B", "C", "D", "col_totals"]
observed = stud_tab.iloc[0:5, 0:2 ]
print(observed)
expected = np.outer(stud_tab["row_totals"][0:5],
stud_tab.loc["col_totals"][0:2]) / 100
print(expected)
chi_squared_stat = (((observed-expected)**2)/expected).sum().sum()
print('Calculated : ',chi_squared_stat)
crit = stats.chi2.ppf(q=0.95, df=4)
print('Table Value : ',crit)
if chi_squared_stat>= crit:
    print('H0 is Accepted ')
else:
    print('H0 is Rejected ')
```

Output

```
= RESTART: C:\College\KIRTI\Msc IT Part 1 Sem 1\Practicals\RESEARCH IN COMPUTING
\Python File\PRACTICAL 4B.py
  Grades  Gender
0        C  Female
1        O  Female
2        C   Male
3        C   Male
4        B  Female
..      ...   ...
95       B   Male
96       D  Female
97       B  Female
98       A   Male
99       B   Male

[100 rows x 2 columns]
   Male  Female
O      11     12
A       9     13
B       7     11
C      10      8
D      12      7
[[11.27 11.73]
 [10.78 11.22]
 [ 8.82  9.18]
 [ 8.82  9.18]
 [ 9.31  9.69]]
Calculated : 3.158915138993211
Table Value : 9.487729036781154
H0 is Rejected
```

PRACTICAL 5

Aim:- Perform testing of hypothesis using Z-test.

Use a Z test if:

- Your sample size is greater than 30. Otherwise, use a t test.
- Data points should be independent from each other. In other words, one data point isn't related or doesn't affect another data point.
- Your data should be normally distributed. However, for large sample sizes (over 30) this doesn't always matter.
- Your data should be randomly selected from a population, where each item has an equal chance of being selected.
- Sample sizes should be equal if at all possible.

H₀ - Blood pressure has a mean of 156 units

Code for one-sample Z test.

```
from statsmodels.stats import weightstats as stests
import pandas as pd
from scipy import stats
df = pd.read_csv("blood_pressure.csv")
df[['bp_before', 'bp_after']].describe()
print(df)
ztest ,pval = stests.ztest(df['bp_before'], x2=None, value=156)
print(float(pval))
if pval<0.05:
    print("reject null hypothesis")
else:
    print("accept null hypothesis")
```

Output

```
= RESTART: C:\College\KIRTI\Msc IT Part 1 Sem 1\Practicals\RESEARCH IN COMPUTING
\Python File\PRACTICAL 5 ONE SAMPLE Z TEST.py
   patient  gender agegrp  bp_before  bp_after
0         1   Male  30-45      143      153
1         2   Male  30-45      163      170
2         3   Male  30-45      153      168
3         4   Male  30-45      153      142
4         5   Male  30-45      146      141
..      ...   ...   ...      ...      ...
115      116  Female  60+      152      152
116      117  Female  60+      161      152
117      118  Female  60+      165      174
118      119  Female  60+      149      151
119      120  Female  60+      185      163

[120 rows x 5 columns]
0.6651614730255063
accept null hypothesis
```

Two-sample Z test

In two sample z-test , similar to t-test here we are checking two independent data groups and deciding whether sample mean of two group is equal or not. H_0 : mean of two group is 0 H_1 : mean of two group is not 0.

H_0 : mean of two group is 0

H_1 : mean of two group is not 0

Code

```
import pandas as pd

from statsmodels.stats import weightstats as stests

df = pd.read_csv("blood_pressure.csv")

df[['bp_before','bp_after']].describe()

print(df)

ztest ,pval = stests.ztest(df['bp_before'], x2=df['bp_after'], value=0,alternative='two-sided')

print(float(pval))
```

if pval<0.05:

 print("reject null hypothesis")

else:

 print("accept null hypothesis")

Output

```
= RESTART: C:\College\KIRTI\Msc IT Part 1 Sem 1\Practicals\RESEARCH IN COMPUTING
\Python File\PRACTICAL 5 TWO SAMPLE Z TEST.py
   patient  gender agegrp  bp_before  bp_after
0         1   Male  30-45        143        153
1         2   Male  30-45        163        170
2         3   Male  30-45        153        168
3         4   Male  30-45        153        142
4         5   Male  30-45        146        141
..      ...   ...   ...   ...   ...
115      116 Female  60+        152        152
116      117 Female  60+        161        152
117      118 Female  60+        165        174
118      119 Female  60+        149        151
119      120 Female  60+        185        163

[120 rows x 5 columns]
0.002162306611369422
reject null hypothesis
|
```

PRACTICAL 6

A. Aim:- Perform testing of hypothesis using One-way ANOVA.

ANOVA Assumptions

- The dependent variable (SAT scores in our example) should be continuous.
- The independent variables (districts in our example) should be two or more categorical groups.
- There must be different participants in each group with no participant being in more than one group. In our case, each school cannot be in more than one district.
- The dependent variable should be approximately normally distributed for each category.
- Variances of each group are approximately equal.

From our data exploration, we can see that the average SAT scores are quite different for each district. Since we have five different groups, we cannot use the t-test, use the 1-way ANOVA test anyway just to understand the concepts.

H0 - There are no significant differences between the groups' mean SAT scores.

$$\mu_1 = \mu_2 = \mu_3 = \mu_4 = \mu_5$$

H1 - There is a significant difference between the groups' mean SAT scores.

If there is at least one group with a significant difference with another group, the null hypothesis will be rejected.

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
data = pd.read_csv("scores.csv")
data.head()
data['Borough'].value_counts()
##### There is no total score column, have to create it.
```

```
#####In addition, find the mean score of the each district across all schools.
data['total_score'] = data['Average Score (SAT Reading)'] + \
data['Average Score (SAT Math)'] + \
data['Average Score (SAT Writing)']
data = data[['Borough', 'total_score']].dropna()
x = ['Brooklyn', 'Bronx', 'Manhattan', 'Queens', 'Staten Island']
district_dict = {}
#Assigns each test score series to a dictionary key
for district in x:
    district_dict[district] = data[data['Borough'] == district]['total_score']
y = []
yerror = []
#Assigns the mean score and 95% confidence limit to each district
for district in x:
    y.append(district_dict[district].mean())
    yerror.append(1.96*district_dict[district].std()/np.sqrt(district_dict[district].shape[0]))
print(district + '_std : {}'.format(district_dict[district].std()))

sns.set(font_scale=1.8)
fig = plt.figure(figsize=(10,5))
ax = sns.barplot(x=x, y=y, yerr=yerror)
ax.set_ylabel('Average Total SAT Score')
plt.show()

print(stats.f_oneway(
    district_dict['Brooklyn'], district_dict['Bronx'], \
    district_dict['Manhattan'], district_dict['Queens'], \
    district_dict['Staten Island']
))
```

```
districts = ['Brooklyn', 'Bronx', 'Manhattan', 'Queens', 'Staten Island']
```

```
ss_b = 0
```

```
for d in districts:
```

```
    ss_b += district_dict[d].shape[0] * \
np.sum((district_dict[d].mean() - data['total_score'].mean())**2)
```

```
ss_w = 0
```

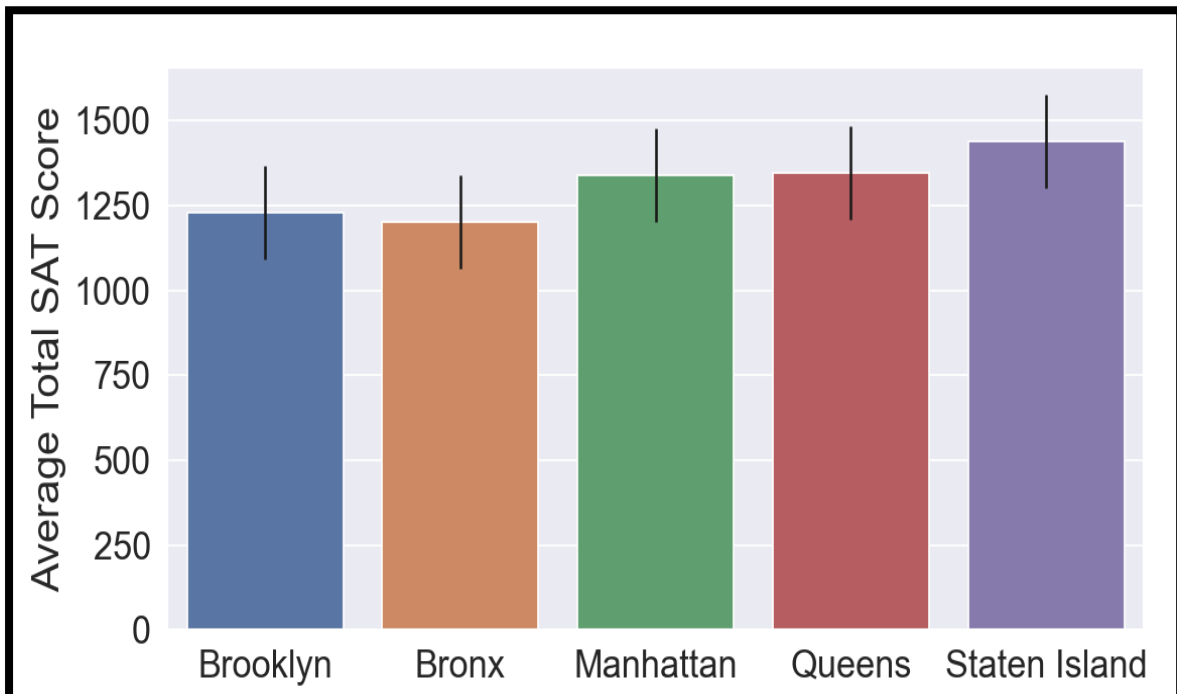
```
for d in districts:
```

```
    ss_w += np.sum((district_dict[d] - district_dict[d].mean())**2)
msb = ss_b/4
msw = ss_w/(len(data)-5)
f=msb/msw
print('F_statistic: {}'.format(f))
```

```
ss_t = np.sum((data['total_score']-data['total_score'].mean())**2)
```

```
eta_squared = ss_b/ss_t
```

```
print('eta_squared: {}'.format(eta_squared))
```


Output

```
= RESTART: C:\College\KIRTI\Msc IT Part 1 Sem 1\Practicals\RESEARCH IN COMPUTING  
\Python File\PRACTICAL 6A.py  
Staten Island_std : 222.30359621222706  
F_onewayResult(statistic=12.733085029201668, pvalue=1.0161974965566023e-09)  
F_statistic: 12.733085029201687  
eta_squared: 0.12099887621529214
```

Since the resulting p-value is less than 0.05, the null hypothesis is rejected and conclude that there is a significant difference between the SAT scores for each district.

B. Aim:- Perform testing of hypothesis using Two-way ANOVA.**Code**

```
import pandas as pd
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.stats.anova import anova_lm
from statsmodels.graphics.factorplots import interaction_plot
import matplotlib.pyplot as plt
from scipy import stats

def eta_squared(aov):
    aov['eta_sq'] = 'NaN'
    aov['eta_sq'] = aov[:-1]['sum_sq']/sum(aov['sum_sq'])
    return aov

def omega_squared(aov):
    mse = aov['sum_sq'][-1]/aov['df'][-1]
    aov['omega_sq'] = 'NaN'
    aov['omega_sq'] = (aov[:-1]['sum_sq']-(aov[:-1]['df']*mse))/(sum(aov['sum_sq'])+mse)
    return aov

datafile = "ToothGrowth.csv"
data = pd.read_csv(datafile)
fig = interaction_plot(data.dose, data.suppl, data.len,
    colors=['red','blue'], markers=['D','^'], ms=10)
N = len(data)
df_a = len(data.suppl.unique()) - 1
df_b = len(data.dose.unique()) - 1
df_axb = df_a*df_b
```

```

df_w = N - (len(data.supp.unique()*len(data.dose.unique()))
grand_mean = data['len'].mean()
#Sum of Squares A – supp
ssq_a = sum([(data[data.supp == l].len.mean()-grand_mean)**2 for l in data.supp])
#Sum of Squares B – dose
ssq_b = sum([(data[data.dose == l].len.mean()-grand_mean)**2 for l in data.dose])
#Sum of Squares Total
ssq_t = sum((data.len - grand_mean)**2)
vc = data[data.supp == 'VC']
oj = data[data.supp == 'OJ']
vc_dose_means = [vc[vc.dose == d].len.mean() for d in vc.dose]
oj_dose_means = [oj[oj.dose == d].len.mean() for d in oj.dose]
ssq_w = sum((oj.len - oj_dose_means)**2) + sum((vc.len - vc_dose_means)**2)
ssq_axb = ssq_t-ssq_a-ssq_b-ssq_w
ms_a = ssq_a/df_a #Mean Square A
ms_b = ssq_b/df_b #Mean Square B
ms_axb = ssq_axb/df_axb #Mean Square AXB
ms_w = ssq_w/df_w
f_a = ms_a/ms_w
f_b = ms_b/ms_w
f_axb = ms_axb/ms_w
p_a = stats.f.sf(f_a, df_a, df_w)
p_b = stats.f.sf(f_b, df_b, df_w)
p_axb = stats.f.sf(f_axb, df_axb, df_w)
results = {'sum_sq':[ssq_a, ssq_b, ssq_axb, ssq_w],
'df':[df_a, df_b, df_axb, df_w],
'F':[f_a, f_b, f_axb, 'NaN'],
'PR(>F)':[p_a, p_b, p_axb, 'NaN']}
columns=['sum_sq', 'df', 'F', 'PR(>F)']
aov_table1 = pd.DataFrame(results, columns=columns,

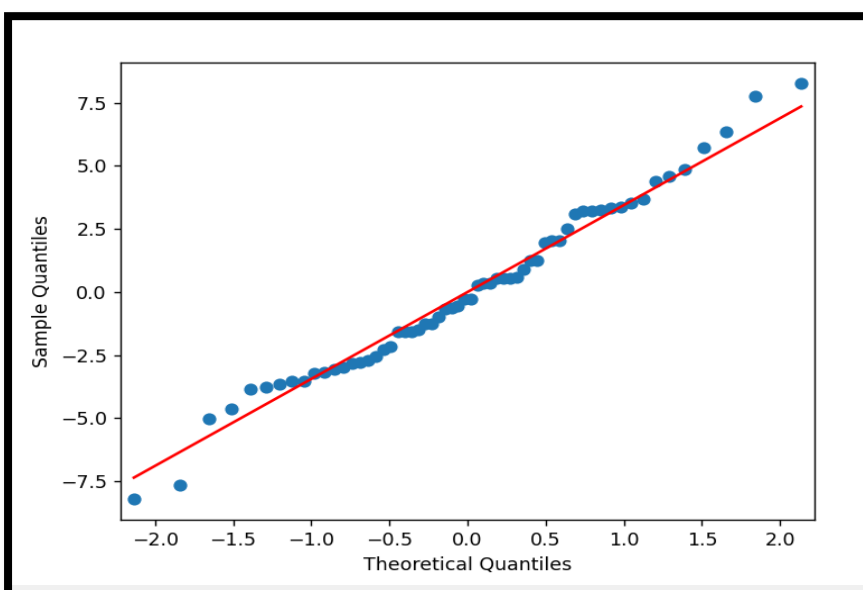
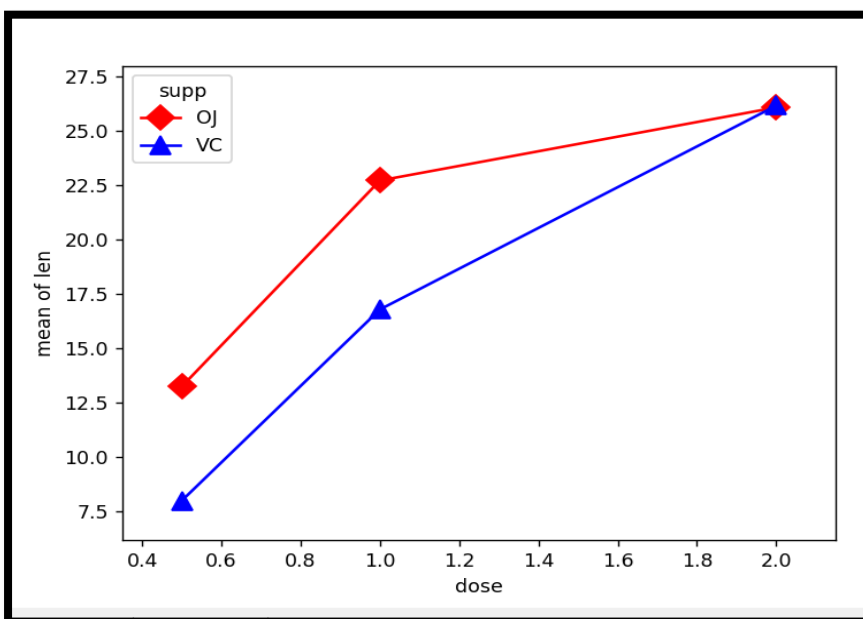
```

```
index=['supp', 'dose',  
      'supp:dose', 'Residual'])  
formula = 'len ~ C(supp) + C(dose) + C(supp):C(dose)'  
model = ols(formula, data).fit()  
aov_table = anova_lm(model, typ=2)  
eta_squared(aov_table)  
omega_squared(aov_table)  
print(aov_table.round(4))  
res = model.resid  
fig = sm.qqplot(res, line='s')  
plt.show()
```

Output

```
= RESTART: C:\College\KIRTI\Msc IT Part 1 Sem 1\Practicals\RESEARCH IN C  
\Python File\PRACTICAL 6B.py
```

	sum_sq	df	F	PR(>F)	eta_sq	omega_sq
C(supp)	205.3500	1.0	15.572	0.0002	0.0595	0.0555
C(dose)	2426.4343	2.0	92.000	0.0000	0.7029	0.6926
C(supp):C(dose)	108.3190	2.0	4.107	0.0219	0.0314	0.0236
Residual	712.1060	54.0	NaN	NaN	NaN	NaN



C. Aim:- Perform testing of hypothesis using MANOVA.

MANOVA is the acronym for Multivariate Analysis of Variance. When analyzing data, we may encounter situations where we have their multiple response variables (dependent variables). In MANOVA there also some assumptions, like ANOVA. Before performing MANOVA we have to check the following assumptions are satisfied or not.

- The samples, while drawing, should be independent of each other.
- The dependent variables are continuous in nature and the independent variables are categorical.
- The dependent variables should follow a multivariate normal distribution.
- The population variance-covariance matrices of each group are same, i.e., groups are homogeneous.

Code

```
import pandas as pd

from statsmodels.multivariate.manova import MANOVA

df = pd.read_csv('iris.csv', index_col=0)
df.columns = df.columns.str.replace(".", "_")
df.head()

print('~~~~~ Data Set ~~~~~')

print(df)

maov = MANOVA.from_formula('Sepal_Length + Sepal_Width + \
Petal_Length + Petal_Width ~ Species', data=df)

print('~~~~~ MANOVA Test Result ~~~~~')

print(maov.mv_test())
```

Output

```

~~~~~ Data Set ~~~~~
Sepal_Length Sepal_Width Petal_Length Petal_Width Species
1           5.1         3.5         1.4         0.2      setosa
2           4.9         3.0         1.4         0.2      setosa
3           4.7         3.2         1.3         0.2      setosa
4           4.6         3.1         1.5         0.2      setosa
5           5.0         3.6         1.4         0.2      setosa
..          ...         ...         ...         ...         ...
146          6.7         3.0         5.2         2.3    virginica
147          6.3         2.5         5.0         1.9    virginica
148          6.5         3.0         5.2         2.0    virginica
149          6.2         3.4         5.4         2.3    virginica
150          5.9         3.0         5.1         1.8    virginica

[150 rows x 5 columns]
~~~~~ MANOVA Test Result ~~~~~
              Multivariate linear model
=====

-----
Intercept      Value  Num DF  Den DF   F Value  Pr > F
-----
Wilks' lambda  0.0170  4.0000 144.0000 2086.7720 0.0000
Pillai's trace  0.9830  4.0000 144.0000 2086.7720 0.0000
Hotelling-Lawley trace 57.9659 4.0000 144.0000 2086.7720 0.0000
Roy's greatest root 57.9659 4.0000 144.0000 2086.7720 0.0000
-----

-----
Species        Value  Num DF  Den DF   F Value  Pr > F
-----
Wilks' lambda  0.0234  8.0000 288.0000 199.1453 0.0000
Pillai's trace  1.1919  8.0000 290.0000  53.4665 0.0000
Hotelling-Lawley trace 32.4773 8.0000 203.4024 582.1970 0.0000
Roy's greatest root 32.1919 4.0000 145.0000 1166.9574 0.0000
=====

```

PRACTICAL 7

A. Aim:- Perform the Random sampling for the given data and analyse it.

Example 1:

From a population of 10 women and 10 men as given in the table in Figure 1 on the left below, create a random sample of 6 people for Group 1 and a periodic sample consisting of every 3rd woman for Group 2. You need to run the sampling data analysis tool twice, once to create Group 1 and again to create Group 2. For Group 1 you select all 20 population cells as the Input Range and Random as the Sampling Method with 6 for the Random Number of Samples. For Group 2 you select the 10 cells in the Women column as Input Range and Periodic with Period 3.

Open existing excel sheet with population data Sample Sheet looks as given below:

	A	B	C	D	E	F	G	H	I	J	K
1	Sr.No	Roll No.	Student's Name	Gender	Grade		Sr. No	Roll No	Student's Name	Gender	Grade
2	1	1	Adhvik	M	O		24	3	Anika	F	A
3	2	2	Aarav	M	O		25	7	Rose	F	O
4	3	5	Aayan	M	A		26	9	Meera	F	C
5	4	13	Sai	M	A		27	11	Aadhya	F	O
6	5	16	Aryan	M	C		28	14	Rea	F	B
7	6	17	Vihaan	M	C		29	25	Aayat	F	O
8	7	34	Rudra	M	O		30	36	Nyra	F	D
9	8	35	Ali	M	O		31	40	Vanya	F	O
10	9	38	Viyaan	M	B		32	41	Shreya	F	O
11	10	42	Yuvaan	M	B		33	46	Mariya	F	O
12	11	43	Darsh	M	O		34	47	Saanvi	F	A
13	12	45	Veer	M	O		35	10	Kiara	F	A
14	13	48	Ivaan	M	D		36	20	Inaya	F	A
15	14	49	Ishaan	M	D		37	21	Pari	F	D
16	15	50	Krishna	M	O		38	72	Ira	F	D
17	16	51	Shivansh	M	A		39	73	Zara	F	O
18	17	54	Athrav	M	A		40	75	Fatima	F	D
19	18	55	Ojas	M	A		41	77	Amayra	F	O
20	19	56	Kiaan	M	O		42	82	Arya	F	D
21	20	59	Pranay	M	O		43	84	Ahana	F	D
22	21	61	Pranav	M	O		44	91	Aarna	F	D
23	22	62	Divit	M	O		45	95	Anaya	F	D
24	23	65	Nishith	M	O		46	4	Prisha	F	C

Set **Cell O1 = Male** and **Cell O2 = Female**

To generate a random sample for male students from given population go to Cell O1 and type
=INDEX(E\$2:E\$62,RANK(B2,B\$2:B\$62))

Drag the formula to the desired no of cell to select random sample.

Now, to generate a random sample for female students go to cell P1 and type

=INDEX(K\$2:K\$40,RANK(H2,H\$2:H\$40))

Drag the formula to the desired no of cell to select random sample.

O	P	
Male	Female	
O	C	
O	D	
O	D	
O	O	
O	D	
A	D	
A	A	
A	A	
O	A	
D	O	
D	O	
O	D	
O	O	
B	D	
B	O	
O	D	
O	O	
C	B	
C	O	
A	C	
A	O	
O	A	
O	D	

B. Aim:- Perform the Stratified sampling for the given data and analyse it.

we are to carry out a hypothetical housing quality survey across Lagos state, Nigeria. And we looking at a total of 5000 houses (hypothetically). We don't just go to one local government and select 5000 houses, rather we ensure that the 5000 houses are a representative of the whole 20 local government areas Lagos state is comprised of. This is called stratified sampling. The population is divided into homogenous strata and the right number of instances is sampled from each stratum to guarantee that the test-set (which in this case is the 5000 houses) is a representative of the overall population. If we used random sampling, there would be a significant chance of having bias in the survey results.

Code

```
import pandas as pd
import numpy as np

import matplotlib
import matplotlib.pyplot as plt

plt.rcParams['axes.labelsize'] = 14
plt.rcParams['xtick.labelsize'] = 12
plt.rcParams['ytick.labelsize'] = 12

import seaborn as sns
color = sns.color_palette()
sns.set_style('darkgrid')

import sklearn
from sklearn.model_selection import train_test_split
housing = pd.read_csv('housing.csv')
print(housing.head())
```

```
print(housing.info())
```

```
#creating a heatmap of the attributes in the dataset
```

```
correlation_matrix = housing.corr()
```

```
plt.subplots(figsize=(8,6))
```

```
sns.heatmap(correlation_matrix, center=0, annot=True, linewidths=.3)
```

```
corr =housing.corr()
```

```
print(corr['median_house_value'].sort_values(ascending=False))
```

```
sns.displot(housing.median_income)
```

```
plt.show()
```

Output

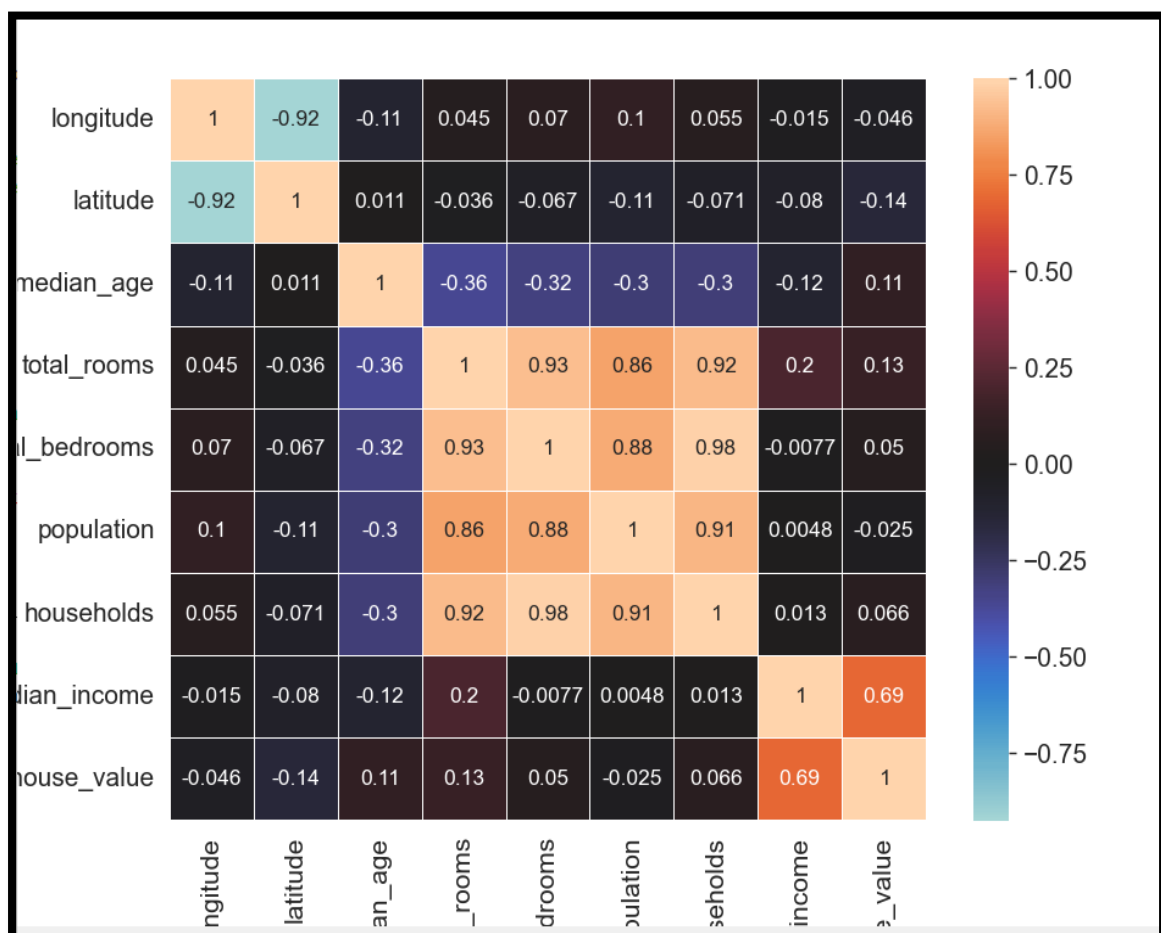
```
= RESTART: C:\College\KIRTI\Msc IT Part 1 Sem 1\Practicals\RESEARCH
\Python File\PRACTICAL 7B.py
longitude latitude ... median_house_value ocean_proximity
0 -122.23 37.88 ... 452600.0 NEAR BAY
1 -122.22 37.86 ... 358500.0 NEAR BAY
2 -122.24 37.85 ... 352100.0 NEAR BAY
3 -122.25 37.85 ... 341300.0 NEAR BAY
4 -122.25 37.85 ... 342200.0 NEAR BAY

[5 rows x 10 columns]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
# Column Non-Null Count Dtype
---
0 longitude 20640 non-null float64
1 latitude 20640 non-null float64
2 housing_median_age 20640 non-null float64
3 total_rooms 20640 non-null float64
4 total_bedrooms 20433 non-null float64
5 population 20640 non-null float64
6 households 20640 non-null float64
7 median_income 20640 non-null float64
8 median_house_value 20640 non-null float64
9 ocean_proximity 20640 non-null object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
None
```

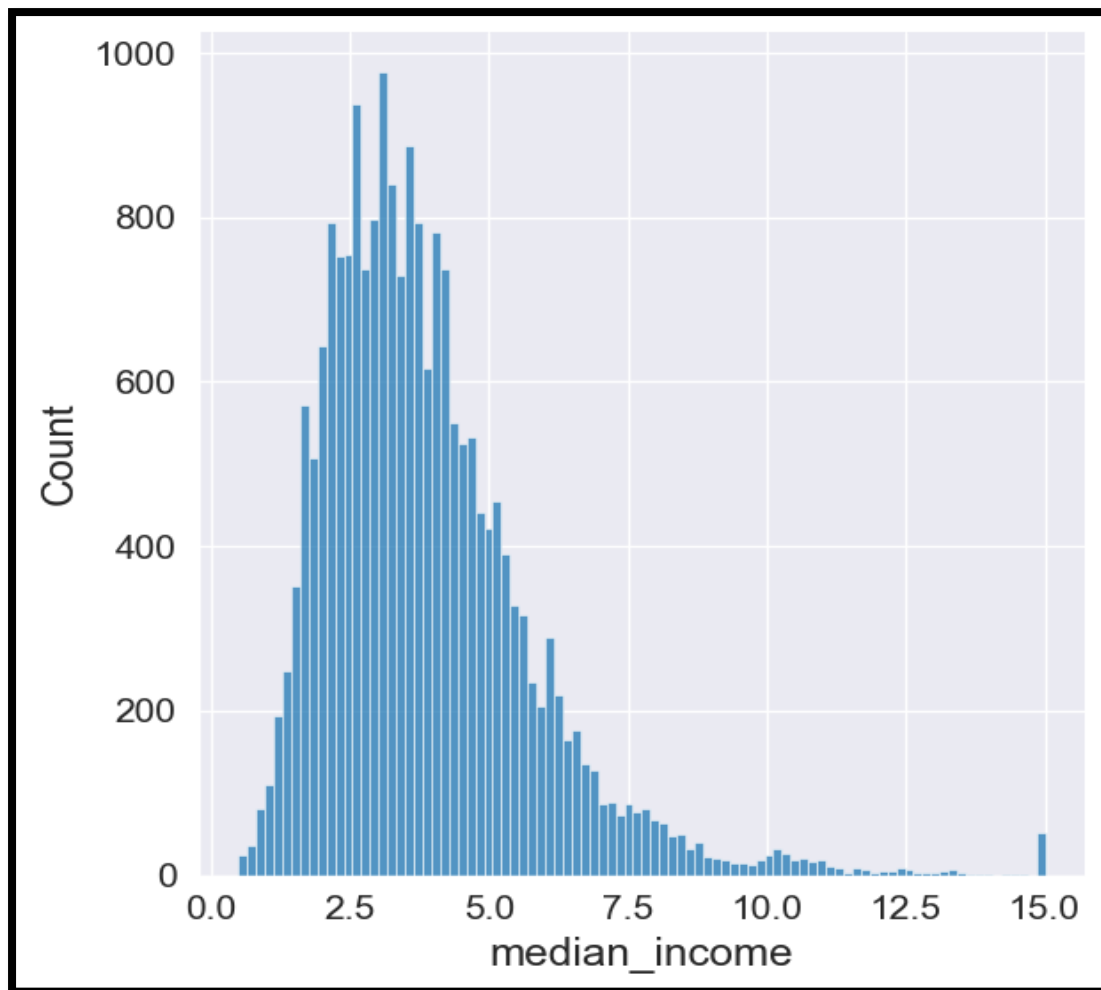
```

median_house_value    1.000000
median_income         0.688075
total_rooms           0.134153
housing_median_age    0.105623
households            0.065843
total_bedrooms        0.049686
population            -0.024650
longitude             -0.045967
latitude              -0.144160
Name: median_house_value, dtype: float64

```



There's a ton of information we can mine from the heatmap above, a couple of strongly positively correlated features and a couple of negatively correlated features. Take a look at the small bright box right in the middle of the heatmap from total_rooms on the left 'y-axis' till households and note how bright the box is as well as the highly positively correlated attributes, also note that median_income is the most correlated feature to the target which is median_house_value.



From the image above, we can see that most median incomes are clustered between \$20,000 and \$50,000 with some outliers going far beyond \$60,000 making the distribution skew to the right.

PRACTICAL 8

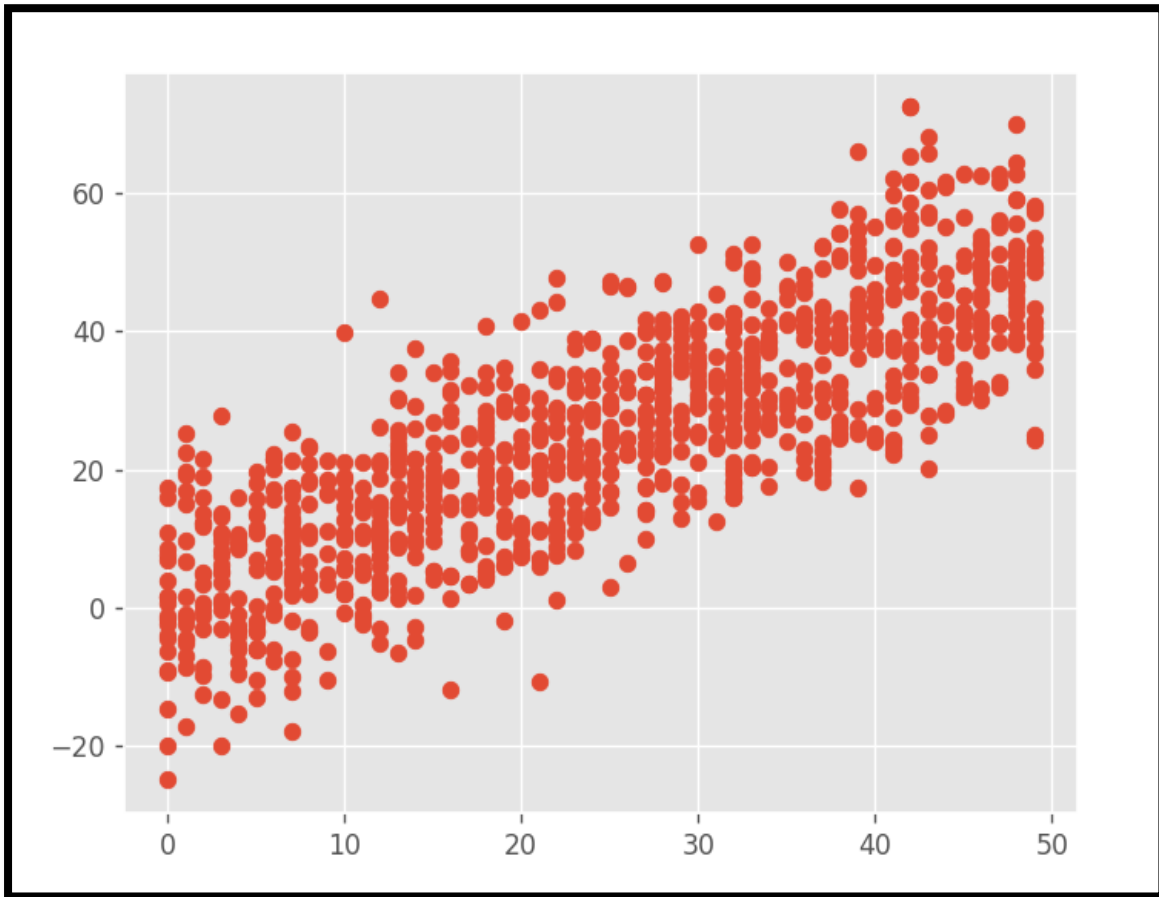
Aim:- Write a program for computing different correlation.

Positive Correlation:

Let's take a look at a positive correlation. Numpy implements a `corrcoef()` function that returns a matrix of correlations of x with x, x with y, y with x and y with y. We're interested in the values of correlation of x with y (so position (1, 0) or (0, 1)).

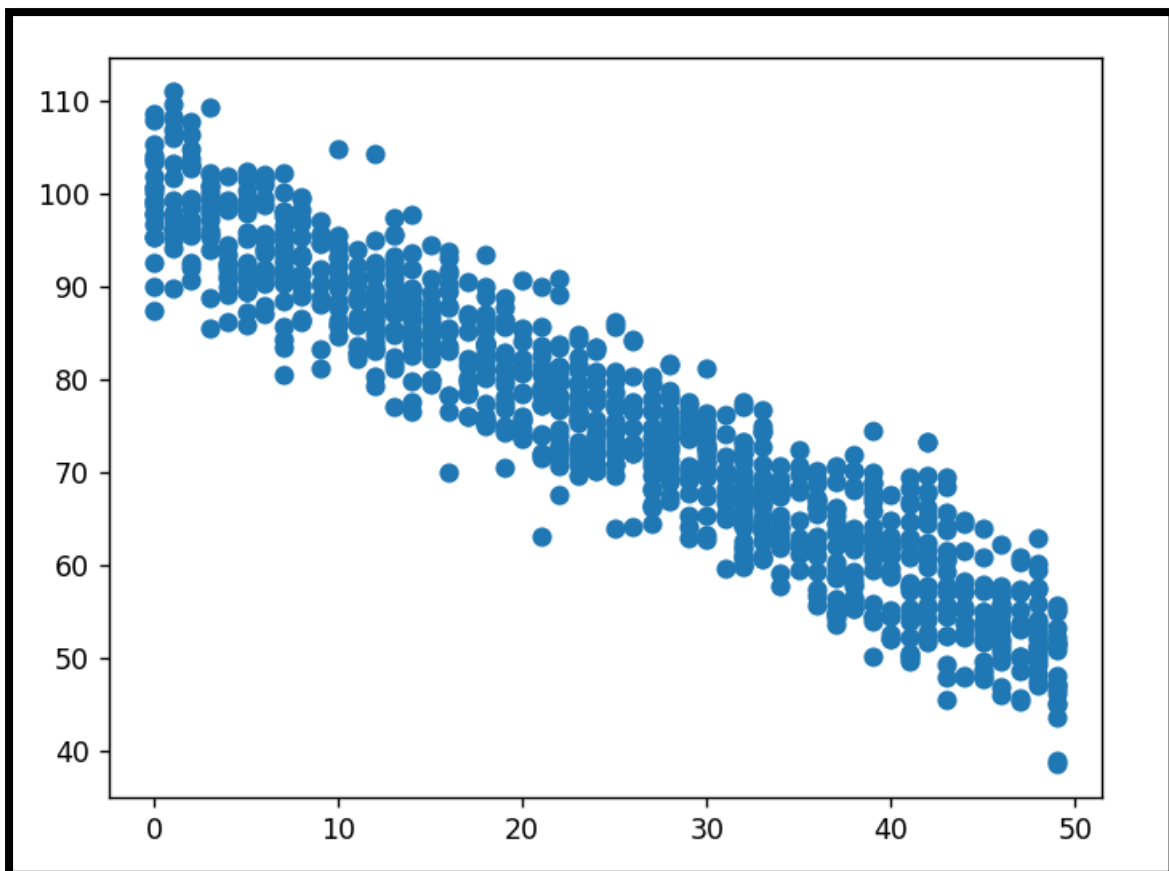
Code

```
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
np.random.seed(1)
# 1000 random integers between 0 and 50
x = np.random.randint(0, 50, 1000)
# Positive Correlation with some noise
y = x + np.random.normal(0, 10, 1000)
np.corrcoef(x, y)
matplotlib.style.use('ggplot')
plt.scatter(x, y)
plt.show()
```

Output

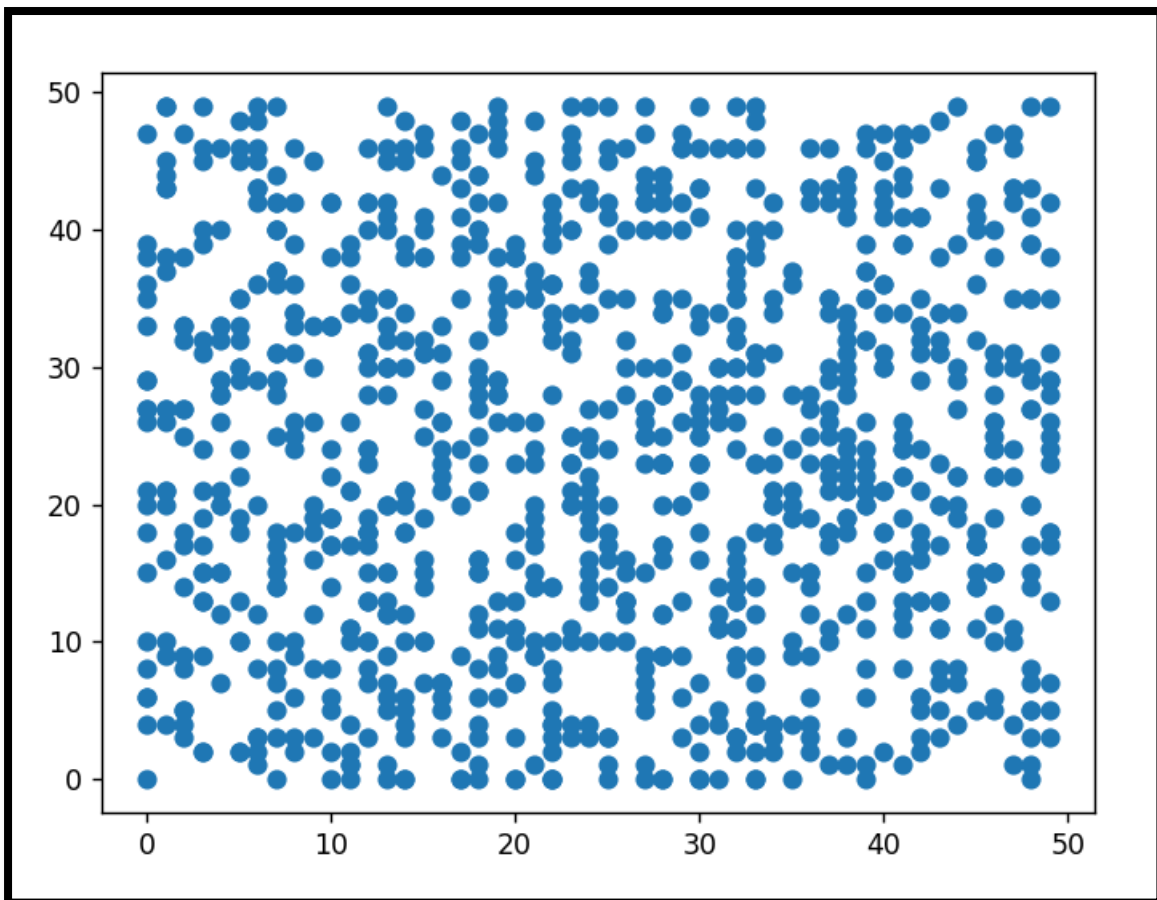
Code for the Negative Correlation:

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(1)
# 1000 random integers between 0 and 50
x = np.random.randint(0, 50, 1000)
# Negative Correlation with some noise
y = 100 - x + np.random.normal(0, 5, 1000)
np.corrcoef(x, y)
plt.scatter(x, y)
plt.show()
```

Output

Code for the No/Weak Correlation:

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(1)
x = np.random.randint(0, 50, 1000)
y = np.random.randint(0, 50, 1000)
np.corrcoef(x, y)
plt.scatter(x, y)
plt.show()
```

Output

PRACTICAL 9

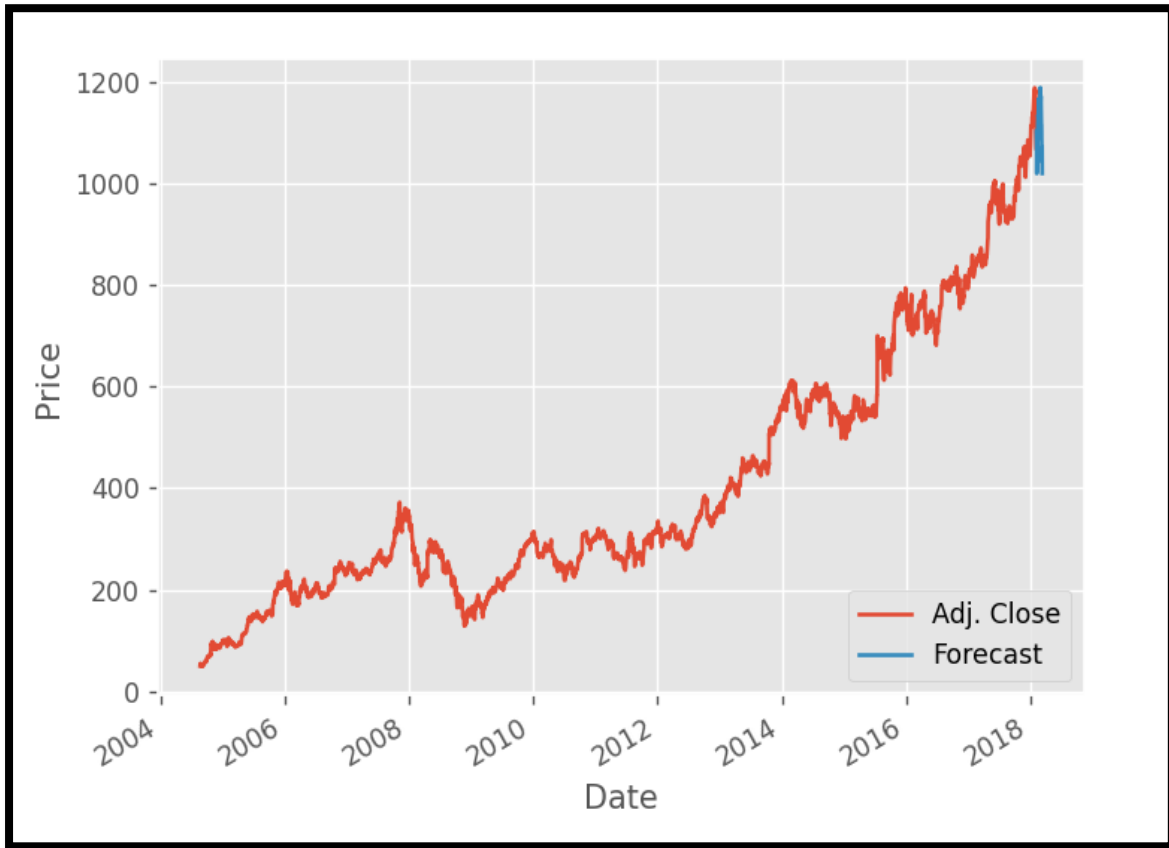
A. Aim:- Write a program to Perform linear regression for prediction.

Code

```
import quandl, math
import numpy as np
import pandas as pd
from sklearn import model_selection, preprocessing, svm
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
import matplotlib.pyplot as plt
from matplotlib import style
import datetime
style.use('ggplot')
df = quandl.get("WIKI/GOOGL")
df = df[['Adj. Open', 'Adj. High', 'Adj. Low', 'Adj. Close', 'Adj. Volume']]
df['HL_PCT'] = (df['Adj. High'] - df['Adj. Low']) / df['Adj. Close'] * 100.0
df['PCT_change'] = (df['Adj. Close'] - df['Adj. Open']) / df['Adj. Open'] * 100.0
df = df[['Adj. Close', 'HL_PCT', 'PCT_change', 'Adj. Volume']]
forecast_col = 'Adj. Close'
df.fillna(value=-99999, inplace=True)
forecast_out = int(math.ceil(0.01 * len(df)))
df['label'] = df[forecast_col].shift(-forecast_out)
X = np.array(df.drop(['label'], 1))
X = preprocessing.scale(X)
X_lately = X[-forecast_out:]
X = X[:-forecast_out]
df.dropna(inplace=True)
```

```
y = np.array(df['label'])

X_train, X_test, y_train, y_test = model_selection.train_test_split(X, y, test_size=0.2)
clf = LinearRegression(n_jobs=-1)
clf.fit(X_train, y_train)
confidence = clf.score(X_test, y_test)
forecast_set = clf.predict(X_lately)
df['Forecast'] = np.nan
last_date = df.iloc[-1].name
last_unix = last_date.timestamp()
one_day = 86400
next_unix = last_unix + one_day
for i in forecast_set:
    next_date = datetime.datetime.fromtimestamp(next_unix)
    next_unix += 86400
    df.loc[next_date] = [np.nan for _ in range(len(df.columns)-1)]+[i]
df['Adj. Close'].plot()
df['Forecast'].plot()
plt.legend(loc=4)
plt.xlabel('Date')
plt.ylabel('Price')
plt.show()
```

Output

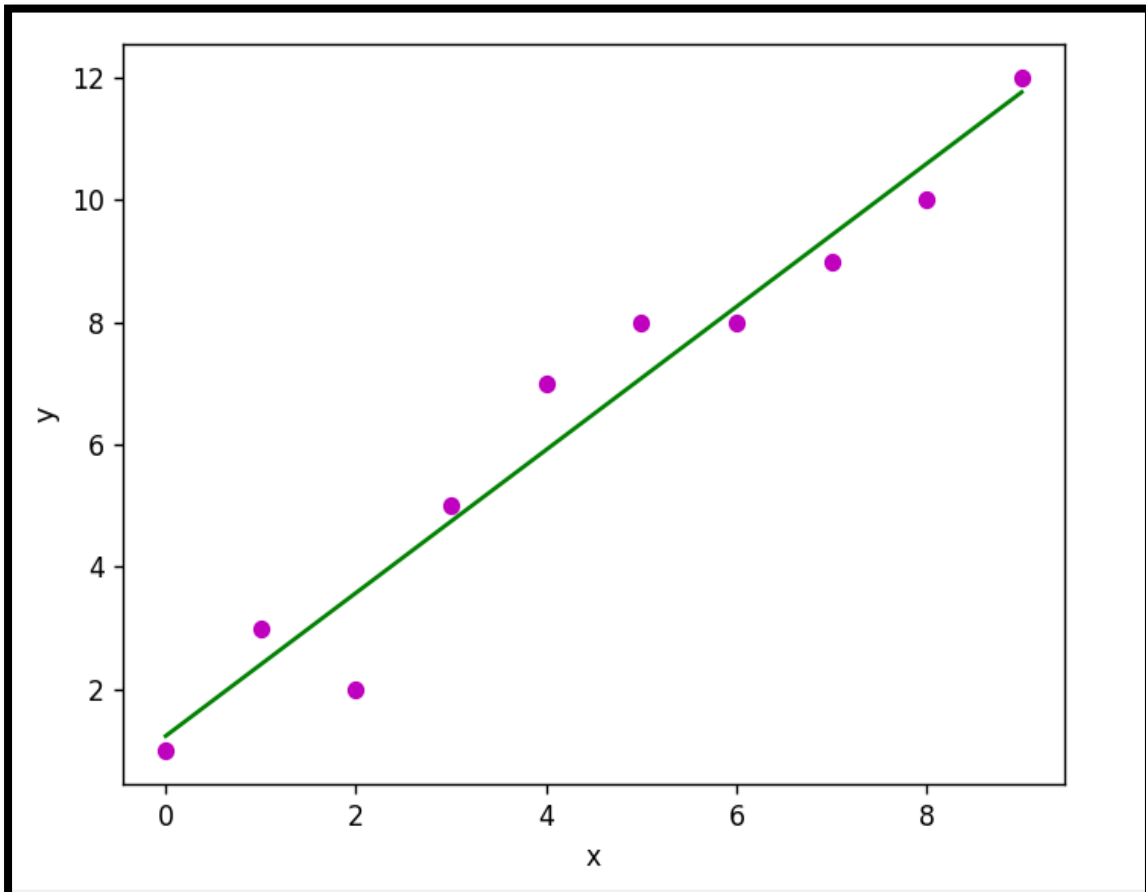
B. Aim:- Perform polynomial regression for prediction.**Code**

```
import numpy as np
import matplotlib.pyplot as plt
def estimate_coef(x, y):
    # number of observations/points
    n = np.size(x)
    # mean of x and y vector
    m_x, m_y = np.mean(x), np.mean(y)
    # calculating cross-deviation and deviation about x
    SS_xy = np.sum(y*x) - n*m_y*m_x
    SS_xx = np.sum(x*x) - n*m_x*m_x
    # calculating regression coefficients
    b_1 = SS_xy / SS_xx
    b_0 = m_y - b_1*m_x
    return (b_0, b_1)
def plot_regression_line(x, y, b):
    # plotting the actual points as scatter plot
    plt.scatter(x, y, color = "m", marker = "o", s = 30)
    # predicted response vector
    y_pred = b[0] + b[1]*x
    # plotting the regression line
    plt.plot(x, y_pred, color = "g")
    # putting labels
    plt.xlabel('x')
    plt.ylabel('y')
    # function to show plot
    plt.show()
```

```
def main():  
    # observations  
    x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])  
    y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])  
    # estimating coefficients  
    b = estimate_coef(x, y)  
    print("Estimated coefficients:\nb_0 = {} b_1 = {}".format(b[0], b[1]))  
    # plotting regression line  
    plot_regression_line(x, y, b)  
if __name__ == "__main__":  
    main()
```

Output

```
= RESTART: C:\College\KIRTI\Msc IT Part 1 Sem 1\Practicals\R  
\Python File\PRACTICAL 9B.py  
Estimated coefficients:  
b_0 = 1.23636363636363 b_1 = 1.16969696969697
```



PRACTICAL 10

A. Aim:- Write a program for multiple linear regression analysis.

Step #1: Data Pre-Processing

- a) Importing The Libraries.
- b) Importing the Data Set.
- c) Encoding the Categorical Data.
- d) Avoiding the Dummy Variable Trap.
- e) Splitting the Data set into Training Set and Test Set.

Step #2: Fitting Multiple Linear Regression to the Training set

Step #3: Predicting the Test set results.

Code

```
import numpy as np
import matplotlib as mpl
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
def generate_dataset(n):
    x = []
    y = []
    random_x1 = np.random.rand()
    random_x2 = np.random.rand()
    for i in range(n):
        x1 = i
        x2 = i/2 + np.random.rand()*n
        x.append([1, x1, x2])
        y.append(random_x1 * x1 + random_x2 * x2 + 1)
    return np.array(x), np.array(y)
x, y = generate_dataset(200)
```



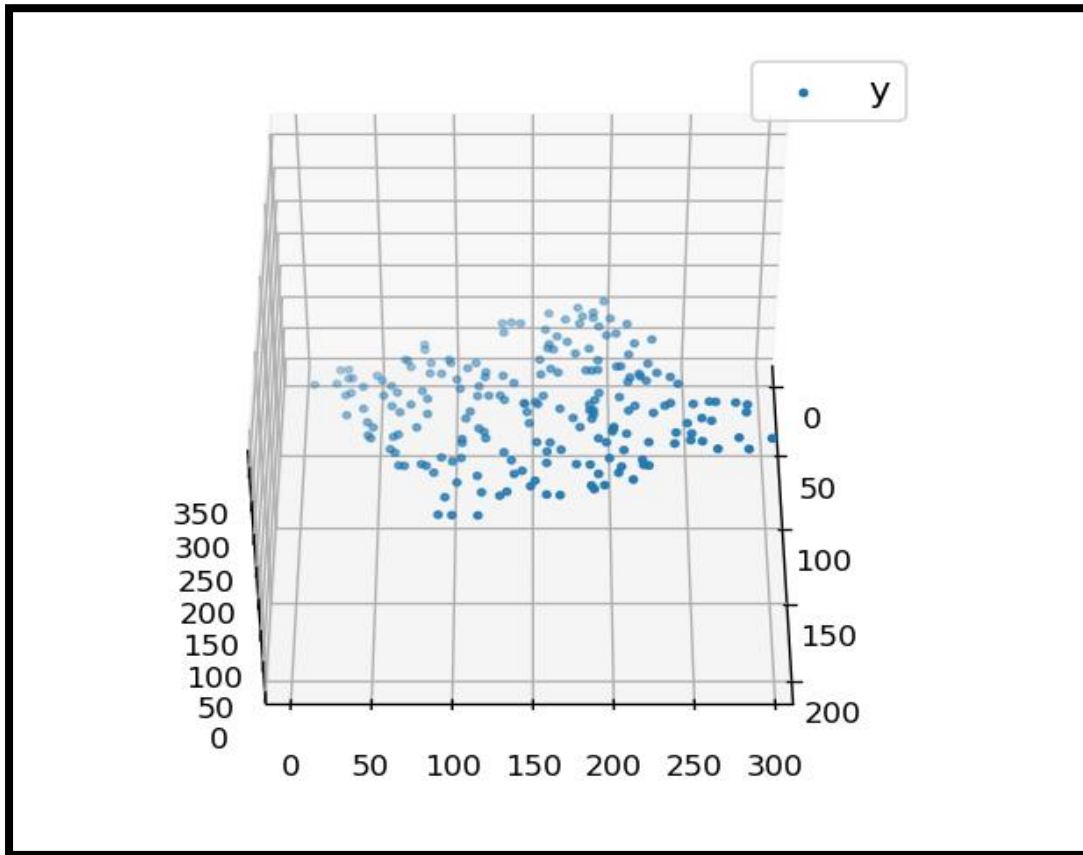
```
mpl.rcParams['legend.fontsize'] = 12
fig = plt.figure()
ax = fig.add_subplot(projection='3d')
ax.scatter(x[:, 1], x[:, 2], y, label='y', s=5)
ax.legend()
ax.view_init(45, 0)
plt.show()

def mse(coef, x, y):
    return np.mean((np.dot(x, coef) - y)**2)/2

def gradients(coef, x, y):
    return np.mean(x.transpose()*(np.dot(x, coef) - y), axis=1)

def multilinear_regression(coef, x, y, lr, b1=0.9, b2=0.999, epsilon=1e-8):
    prev_error = 0
    m_coef = np.zeros(coef.shape)
    v_coef = np.zeros(coef.shape)
    moment_m_coef = np.zeros(coef.shape)
    moment_v_coef = np.zeros(coef.shape)
    t = 0
    while True:
        error = mse(coef, x, y)
        if abs(error - prev_error) <= epsilon:
            break
        prev_error = error
        grad = gradients(coef, x, y)
        t += 1
        m_coef = b1 * m_coef + (1-b1)*grad
        v_coef = b2 * v_coef + (1-b2)*grad**2
        moment_m_coef = m_coef / (1-b1**t)
        moment_v_coef = v_coef / (1-b2**t)
        delta = ((lr / moment_v_coef**0.5 + 1e-8) *
```

```
(b1 * moment_m_coef + (1-b1)*grad/(1-b1**t)))  
coef = np.subtract(coef, delta)  
return coef  
coef = np.array([0, 0, 0])  
c = multilinear_regression(coef, x, y, 1e-1)  
fig = plt.figure()  
ax = fig.gca(projection='3d')  
ax.scatter(x[:, 1], x[:, 2], y, label='y', s = 5, color="dodgerblue")  
ax.scatter(x[:, 1], x[:, 2], c[0] + c[1]*x[:, 1] + c[2]*x[:, 2], label='regression', s = 5, color="orange")  
ax.view_init(45, 0)  
ax.legend()  
plt.show()
```

Output

B. Aim:- Perform logistic regression analysis.

Logistic regression is a classification method built on the same concept as linear regression. With linear regression, we take linear combination of explanatory variables plus an intercept term to arrive at a prediction.

In this example we will use a logistic regression model to predict survival

Code

```
import os

import numpy as np
import pandas as pd
import matplotlib

import matplotlib.pyplot as plt
import scipy.stats as stats

from sklearn import linear_model
from sklearn import preprocessing
from sklearn import metrics

matplotlib.style.use('ggplot')
plt.figure(figsize=(9,9))

def sigmoid(t): # Define the sigmoid function
    return (1/(1 + np.e**(-t)))

plot_range = np.arange(-6, 6, 0.1)
y_values = sigmoid(plot_range)

# Plot curve
plt.plot(plot_range, # X-axis range
         y_values, # Predicted values
         color="red")

titanic_train = pd.read_csv("titanic_train.csv") # Read the data
char_cabin = titanic_train["Cabin"].astype(str) # Convert cabin to str
```

```
new_Cabin = np.array([cabin[0] for cabin in char_cabin]) # Take first letter
titanic_train["Cabin"] = pd.Categorical(new_Cabin) # Save the new cabin var
# Impute median Age for NA Age values
new_age_var = np.where(titanic_train["Age"].isnull(), # Logical check
    28, # Value if check is true
    titanic_train["Age"]) # Value if check is false
titanic_train["Age"] = new_age_var
label_encoder = preprocessing.LabelEncoder()
# Convert Sex variable to numeric
encoded_sex = label_encoder.fit_transform(titanic_train["Sex"])
# Initialize logistic regression model
log_model = linear_model.LogisticRegression()
# Train the model
log_model.fit(X = pd.DataFrame(encoded_sex),
    y = titanic_train["Survived"])
# Check trained model intercept
print(log_model.intercept_)
# Check trained model coefficients
print(log_model.coef_)
# Make predictions
preds = log_model.predict_proba(X= pd.DataFrame(encoded_sex))
preds = pd.DataFrame(preds)
preds.columns = ["Death_prob", "Survival_prob"]
# Generate table of predictions vs Sex
pd.crosstab(titanic_train["Sex"], preds.loc[:, "Survival_prob"])
# Convert more variables to numeric
encoded_class = label_encoder.fit_transform(titanic_train["Pclass"])
encoded_cabin = label_encoder.fit_transform(titanic_train["Cabin"])
train_features = pd.DataFrame([encoded_class,
    encoded_cabin,
```

```
encoded_sex,
titanic_train["Age"])).T
# Initialize logistic regression model
# Train the model
log_model.fit(X = train_features ,
y = titanic_train["Survived"])
# Check trained model intercept
print(log_model.intercept_)
# Check trained model coefficients
print(log_model.coef_)
# Make predictions
preds = log_model.predict(X= train_features)
# Generate table of predictions vs actual
pd.crosstab(preds,titanic_train["Survived"])
log_model.score(X = train_features ,
y = titanic_train["Survived"])
metrics.confusion_matrix(y_true=titanic_train["Survived"], # True labels
y_pred=preds) # Predicted labels
# View summary of common classification metrics
print(metrics.classification_report(y_true=titanic_train["Survived"],
y_pred=preds) )
# Read and prepare test data
titanic_test = pd.read_csv("titanic_test.csv") # Read the data
char_cabin = titanic_test["Cabin"].astype(str) # Convert cabin to str
new_Cabin = np.array([cabin[0] for cabin in char_cabin]) # Take first letter
titanic_test["Cabin"] = pd.Categorical(new_Cabin) # Save the new cabin var
# Impute median Age for NA Age values
new_age_var = np.where(titanic_test["Age"].isnull(), # Logical check
28, # Value if check is true
titanic_test["Age"]) # Value if check is false
```

```
titanic_test["Age"] = new_age_var

# Convert test variables to match model features
encoded_sex = label_encoder.fit_transform(titanic_test["Sex"])
encoded_class = label_encoder.fit_transform(titanic_test["Pclass"])
encoded_cabin = label_encoder.fit_transform(titanic_test["Cabin"])
test_features = pd.DataFrame([encoded_class,
                              encoded_cabin, encoded_sex, titanic_test["Age"]]).T

# Make test set predictions
test_preds = log_model.predict(X=test_features)

# Create a submission for Kaggle
submission = pd.DataFrame({"PassengerId":titanic_test["PassengerId"],
                           "Survived":test_preds})

# Save submission to CSV
submission.to_csv("tutorial_logreg_submission.csv",
                  index=False) # Do not save index values
print(pd)

log_model = linear_model.LogisticRegression()
```

Output

```
= RESTART: C:\College\KIRTI\Msc IT Part 1 Sem 1\Practicals\RESEARCH IN
COMPUTING\Python File\Pract 10 B.py
[1.01628767]
[[-2.44597988]]
[3.85818122]
[[-0.93272694 -0.09865307 -2.51826528 -0.03339524]]
      precision    recall  f1-score   support

         0         0.84        0.83        0.84         549
         1         0.74        0.74        0.74         342

 accuracy          0.80          891
 macro avg         0.79         0.79         0.79          891
weighted avg         0.80         0.80         0.80          891

<module 'pandas' from 'C:\\Users\\Ravi yadav\\AppData\\Local\\Programs
\\Python\\Python310\\lib\\site-packages\\pandas\\__init__.py'>
|
```